

# Current status and separate repositories

Updated 10 October 2026. This section supersedes conflicting older examples. The following earlier handbook remains a historical learning reference, not proof that the new delivery worker is active.

## Contents

1. Repositories and processes 2. Calendar web and shared dates 3. Pending activation

## Repositories

Music/radio/private server panel: DoubleP987/tutel-hub. Calendar web/OAuth/groups/delivery APIs: DoubleP987/tutel-calendar. Dependencies, secrets, databases and sessions are independent. A Calendar OAuth application does not start a second Discord bot process.

## Calendar web

Groups, single-action invitations, mobile sidebar and shared general dates are deployed on homeserver. Shared dates are stored once, never copied per group. Group activities remain isolated. Indexed common reads support display and group reminder scheduling.

## Pending work

Production Discord worker and private panel bridge cutover remain pending. Simultaneous multi-group display and explicit Discord-link activity viewing are not complete. Actual email receipt and device Push are not verified end-to-end. Website hosting does not fail over to Oracle. Calendar CURRENT-STATUS.md and its separate user guides are the current operational reference.

# Multi-group Calendar - implementation guide

Local revision: 10 October 2026. Source implemented; not deployed or runtime-tested in this revision. Existing production behavior is described separately in DEPLOYMENT.md. This guide supersedes earlier single-calendar assumptions.

## 1. Application boundaries

Calendar owns accounts, groups, membership, events, invitations, preferences, inbox records, Discord channel bindings and delivery jobs. Tutel Bot owns music/radio, Discord connections, host election and the private server control panel. They communicate through authenticated HTTP APIs; they never open the same SQLite file.

Browser -> Calendar session + CSRF -> group permission -> Calendar SQLite

Private panel -> server-side control key -> Calendar management API

Active Discord bot -> delivery key -> claim jobs -> Discord -> acknowledge

An account has many memberships; every membership has its own role, suspension, trust and preferences. Each event and Discord binding has a group\_id. Knowing another group's ID is not permission to read or modify it. Music remains independent of Calendar availability.

## 2. Groups and roles

Sign in, choose Create group, enter a name and unique slug. New groups are private. Switch groups using the selector; the last group and signed-in display/reminder preferences are saved in SQLite. Guest filters are stored separately for each group in browser storage.

Role / Capabilities

Member / Read group events, submit proposals, edit own pending proposals, set personal reminders

Editor / Member capabilities; edit native published events; treated as trusted under trusted approval

Manager / Review proposals, manage ordinary members/trust, invitations, group reminder settings and existing Discord bindings

Owner / Manager capabilities plus appoint managers, change visibility/name and transfer ownership

Platform administrator / Manage all groups through the private panel; separate from group roles

Private groups require membership. Public groups permit read-only viewing through /g/<slug>. There is no public group directory, search or membership request flow. Blocking a membership does not block other groups. Owners must transfer ownership before their role can be removed or blocked. Self-removal of management access is rejected.

Google/Discord bootstrap administrator IDs and migrated original managers are privileged Calendar accounts. Review that bootstrap list before production migration. Calendar roles do not create private bot-panel accounts or server credentials.

### 3. Invitations and preassigned people

Owners/managers can create a link or join code with 1–30 days validity and 1–500 uses; defaults are seven days and one use. Codes contain 12 random hexadecimal characters. Tokens are stored as SHA-256 hashes; plaintext links are shown only when created. Revoke unused invitations from the invitation list. Share private links deliberately.

An email invitation is assigned to an exact normalized provider-verified address, one use only. It creates a pending invitation, not a fabricated user account. When the invited person signs in, matching invitations are claimed. Existing members can refresh to claim newly created invitations. An invitation does not overwrite an existing role or bypass a membership suspension.

CSV supports UTF-8/BOM, quoted fields and CRLF. Preview before confirmation. Maximum 500 rows and 100 KB; duplicate or invalid email rows prevent the entire import. Roles in CSV are member/editor only. Example:

email,name,role

student@parichat.skru.ac.th,Student,member

editor@example.com,Editor,editor

Preassignment needs no mail server. Actual invitation emails require SMTP\_HOST, SMTP\_PORT, SMTP\_FROM and, when required by the provider, SMTP\_USER/SMTP\_PASSWORD. The sender uses TLS and Nodemailer; the UI has an explicit send email checkbox. CSV imports create preassignments without bulk mail. A configured SMTP service, mail delivery and spam reputation are external prerequisites; no email account has been provisioned by this revision.

### 4. Accounts and OAuth

Google uses basic OpenID/profile/email scopes. University Google accounts are supported; verified Gmail addresses or a provider-verified hosted-domain account can claim email invitations. Discord now requests identify email; users may need to consent again. Never trust a user-typed email for automatic membership.

From account settings, link the other provider while signed in. A fresh OAuth callback and the current session prove both accounts. If two existing accounts are linked, memberships/events/notifications/subscriptions move transactionally to the current account; higher roles survive, suspensions are retained, and the other account's sessions are revoked. Similar display names or emails alone never merge accounts.

OAuth state is single-use with expiry; return paths are local-only and retain the group/date/event. Session cookies remain HttpOnly/SameSite=Lax and Secure on HTTPS. Browser writes require origin and CSRF checks. Service APIs use separate credentials, not browser cookies.

### 5. Discord group binding

After activating group integration, run /calendar connect in the target channel. The requester needs Manage Server; the bot needs View Channel, Send Messages and Embed Links. The bot obtains a single-use ten-minute connection link. Open it, sign in/link the same Discord account, choose a group you manage and confirm. The grant binds that exact guild/channel, so browser input cannot substitute a different channel.

The private system panel can also create a binding after checking the bot's channel permissions. Group managers may configure or disable an existing binding, but cannot arbitrarily change its Discord destination.

One group can have several bindings; a channel can receive more than one group's agenda. A private group's reminder is visible to everyone with access to its configured Discord channel: check Discord permissions before binding.

Default summary is 07:00 Asia/Bangkok; tomorrow's summary is noon the preceding day. Timed reminders are optional; category filters, lead time and embed color are per binding. The calendar link includes the exact group/date, and event reminders also include the event ID. Private links still require member login. Daily events are combined into an agenda, not one message per event. Up to five group agendas fit one message; larger batches split to respect Discord button/embed limits, and long agendas link to the full list.

The explicit send test message button queues a diagnostic job. It does not prove delivery until the active bot receives it and a message ID is recorded.

## 6. Delivery reliability and host switching

Only the active elected bot polls, every 15 seconds. Calendar creates stable per-binding deduplication keys, atomically claims jobs for 120 seconds, records delivery start, and receives acknowledgement with the Discord message ID. A bot-host change does not change Calendar ownership or copy its database.

State / Meaning

pending / Available to claim

claimed / Worker owns a temporary lease; safe to reclaim after expiry if sending has not started

sending / Sending began; another host must not blindly send again

sent / Message ID acknowledged

failed / Known failure such as unavailable destination; inspect and manually retry

uncertain / Worker stopped or network/acknowledgement failed after sending began; check the channel before retry

expired / Delivery window has passed

New scheduled jobs have a five-minute catch-up window. Pending/claimed jobs expire after one hour.

Exactly-once delivery across two independent services is not guaranteed: a crash between Discord acceptance and local acknowledgement can be uncertain. The panel exposes that state and requires confirmation that the channel was checked before manually retrying. Bot-side message IDs and Discord nonces reduce duplicates. No endless automatic retry flood is used.

If Calendar itself is offline, music can continue and a standby bot can take over music, but neither bot can fetch new Calendar jobs. The website/database do not automatically fail over to Oracle in this implementation. A calendar replica/backup restoration plan is a separate infrastructure task.

## 7. Main private control panel

Open Manage all groups using an existing bot-panel administrator account. It displays overview counts, groups, accounts/memberships, group members/roles/trust, ownership transfer, invitations/CSV, native event editing/review, channel bindings/test messages, delivery jobs/retry and audit history. Imported events are read-only while the legacy source remains authoritative.

The browser calls the bot's private proxy. The proxy checks session, admin role, CSRF and a fixed path allowlist, then calls Calendar with CALENDAR\_CONTROL\_SECRET server-side. Browser JavaScript and Discord messages never receive that credential. CALENDAR\_BOT\_SECRET has a smaller API surface for bindings, connection grants and delivery claims; it cannot manage users or events. Use distinct long keys and protect the private panel with Tailscale/LAN access.

Lists are bounded: up to 1,000 users/events, 500 invitations/jobs/audit entries. This is a small-deployment interface, not an unlimited analytics export.

## 8. Local setup and migration

Node.js 24+ is required. Install dependencies separately in D:/tutel-calendar and D:/tutel-bot. Existing local .env files contain matching, randomly generated service credentials; examples contain blanks. Do not overwrite a configured .env with the example.

Calendar .env needs both CALENDAR\_CONTROL\_SECRET and CALENDAR\_BOT\_SECRET. Bot .env needs the same respective values plus:

```
CALENDAR_SERVICE_URL=http://127.0.0.1:3100
```

```
CALENDAR_WEB_URL=http://127.0.0.1:3100
```

```
CALENDAR_GROUPS_ENABLED=0
```

HTTP is permitted only for loopback development. For separate hosts use a trusted HTTPS Calendar URL accessible from both bot hosts. Real OAuth mode requires HTTPS and ENABLE\_DEMO=0; local demo stays loopback-only. Start Calendar with npm start, then open http://127.0.0.1:3100. The main panel uses its existing process/configuration. Running the real bot requires its existing Discord and database setup.

Keep CALENDAR\_GROUPS\_ENABLED=0 during migration: the old reminder scheduler continues. After creating and checking group/channel bindings, set it to 1 on both bot hosts, register the command definitions, and restart the services in a coordinated cutover. This flag switches the scheduler globally, not per group. Do not enable it while unmigrated channels still depend on the old scheduler. New-mode calendar commands use the web workflow; legacy add/remove/list command behavior is not preserved in new mode.

Back up Calendar SQLite with a consistent SQLite backup or while stopped; a live WAL database may have committed rows in its WAL file. Keep the bot's own backup separately. Old event IDs remain, events migrate into the public cs-skru group, original members join that group once, and original settings are copied. New groups do not inherit those events. Review original managers before migrating administrator privileges. Stop legacy import/publishing only after Calendar becomes authoritative; do not introduce a two-way sync loop. Imported source removal is not yet reconciled automatically.

No production services, OAuth dashboards, DNS, GitHub or deployments were changed for this local revision. No runtime or cross-group isolation tests were run; complete a staging check before production activation.

## 9. Source and API map

Module / Responsibility

groups/schema.js / Additive schema migration, legacy group bootstrap, audit

groups/access.js / Resolve group and enforce read/member/manager/owner guards

groups/invitations.js / Tokens, email matching, CSV parsing, atomic accept

groups/routes.js / Group creation/selection, invitations, ownership transfer

auth/oauth.js, auth/link-accounts.js / Login, verified identities, safe account linking

events/routes.js / Group-scoped event validation, proposal/review/edit

members/ / Membership roles and personal/group settings

notifications/ / Group-scoped inbox, Web Push, reminders and optional SMTP

integrations/discord/ / Bindings, restricted service key, leased delivery jobs

integrations/control/ / Platform management API with separate service key

public/groups.js / Group selector, invitations, CSV, account linking, Discord setup

bot integrations/calendar/ / HTTP client and active-host delivery worker

bot web/routes/calendar-admin.js / Private panel proxy and authorization

Browser group routes include /api/groups, /api/group, /api/group/select, /api/invitations, /api/invitations/csv, /api/invitations/accept, /api/accounts/link/:provider, /api/discord-connect and /api/discord-bindings.

Group-scoped requests send x-calendar-group; this is a selector, not authorization.

Bot service routes are under /internal/v1/bot: bindings, connect-grants, claim, jobs/:id/start and jobs/:id/ack.

Management routes are under /internal/v1/control: overview, groups, users, group

members/transfer/invitations/bindings/events, jobs/retry and audit. Exact methods and validation live in the route modules. Never put either service key in a public frontend environment variable.

# Project boundaries and source structure /

## ขอบเขตและโครงสร้าง

คู่มือภาษาไทย

Updated 10 October 2026. This document describes the local refactor, which has not been deployed to homeserver or Oracle. Production continues using the previously deployed file layout.

### Two independent applications

Application / Owns / Does not own

Tutel Bot (D:/tutel-bot) / Discord client, music/radio, guild music settings, host failover, private server control panel / New Calendar members and OAuth sessions

Tutel Calendar (D:/tutel-calendar) / Public calendar, proposals/approval, Google/Discord identities, member roles/preferences, SQLite and Web Push / Server control permissions or the music queue

Each application has its own package.json, package-lock.json, environment, process and data directory. Do not copy secrets, sessions or database files into the other application's repository. Calendar management roles never grant access to the bot control panel. SQLite is suitable for the current single Calendar host; it is not a shared file for concurrent homeserver/Oracle writes.

### Current integration, not the future design

Original bot calendar + legacy reminder scheduler

-> Vercel snapshot API

-> Calendar legacy importer (every 60 seconds)

New Calendar proposals -> approval -> Calendar SQLite (not yet connected back to the original Discord reminder scheduler)

The bot's original calendar implementation now resides in src/integrations/calendar/legacy/. It still runs and keeps original guild configuration, daily summaries, links and publishing compatible. Relocating those files does not migrate data or retire the scheduler. The importer is src/integrations/legacy-calendar/importer.js in the new Calendar project. Imported data is read-only in the Calendar member workflow; disappearance from the source snapshot does not yet delete imported rows.

### Intended integration after a separate migration

Calendar should become the single owner of events and reminder schedules. The active bot should pull authenticated delivery jobs, claim a job, send to Discord and acknowledge it. Jobs require stable IDs, atomic claims/expiry, retries and recorded Discord message IDs. Cross-service authentication must be separate from browser sessions and the bot login token. No such delivery endpoint is implemented by this source-only refactor.

Before switching: back up data, preserve legacy identifiers and deep links, reconcile categories/time zones/recurrence, migrate channel subscriptions, compare generated summaries, then disable the old scheduler

and publisher. Stop legacy import once Calendar is authoritative; never create a synchronization loop. Retain rollback data until verification is complete. Music should remain independent of Calendar outages.

## **Bot source map**

src/

index.js application composition and shutdown

auth/ private panel accounts and sessions

bot/ Discord lifecycle and private replies

commands/ interaction handlers

music/ playback, streams, queue, panels, radio

cluster/ election, relay and host failover

config/ bot settings and profile

database/ shared bot store and settings

integrations/calendar/legacy/ original calendar and Discord reminders

web/ private control panel and legacy routes

Music, authentication and panel settings import their owners directly from database/ and auth/. The former calendar database facade has been removed. Calendar commands and old calendar panel routes remain integration clients until data migration. The promotion website stays in promo-site/; its deployment is independent of the bot process. vercel-public/ and netlify-public/ are retained legacy hosting assets, not the new Calendar frontend.

## **Calendar source map**

src/

server.js composition, middleware, lifecycle

auth/oauth.js provider login, sessions and guards

members/routes.js member roles and calendar settings

members/preferences.js saved display/reminder preferences

events/routes.js proposal validation and review

events/categories.js category catalog and defaults

database/connection.js SQLite connection lifecycle

database/migrations.js idempotent schema initialization/upgrades

database/demo-fixtures.js local preview records only

notifications/service.js inbox routes and Web Push delivery

notifications/scheduler.js background reminder scheduling

integrations/legacy-calendar/importer.js original snapshot adapter

http/security.js request limits and origin protection

public/ browser UI, styles, service worker

scripts/ explicit maintenance utilities

integration/ service/tunnel deployment examples

Migrations remain additive and idempotent, using the existing schema. This refactor does not create a new database or change user/event IDs. Demo fixtures stay gated by `ENABLE_DEMO` and production remains disabled. Route modules still include their application logic where it is small; split services/repositories when behavior grows, rather than creating empty abstraction layers.

## Formatting and maintenance

Both projects use Prettier, two-space indentation, semicolons, single quotes in JavaScript, LF line endings and a 100-column target. Run `npm run format` to write formatting or `npm run format:check` to check it when requested. Formatter ignore files exclude credentials, databases, dependencies and generated snapshots. Formatting does not establish behavioral correctness.

Read dependencies from `package.json`. Keep database access out of browser code. Keep provider secrets server-only. Log useful failure context without tokens or session values. Add migration and API-contract documentation when behavior changes. Documentation source paths are updated; earlier handbook code excerpts are learning snapshots, not files to copy blindly.

## ภาษาไทย

แยกโปรเจกต์และฐานข้อมูลแล้วในระดับโครงสร้าง แต่ระบบเดิมยังต้องอยู่ระหว่างย้ายข้อมูล จึงรวบรวมไว้ใน `integrations/calendar/legacy` อย่างชัดเจน การจัดไฟล์เดอริไม่ได้ทำให้กิจกรรมใหม่ส่ง Discord ได้ทันที ขั้นตอนต่อไปต้องย้ายเจ้าของข้อมูลและทำ API งานแจ้งเตือนพร้อมป้องกันส่งซ้ำ งานรอบนี้ปรับเฉพาะไฟล์ในเครื่อง ไม่มีการ deploy หรือ push GitHub และไม่ได้รันทดสอบระบบ

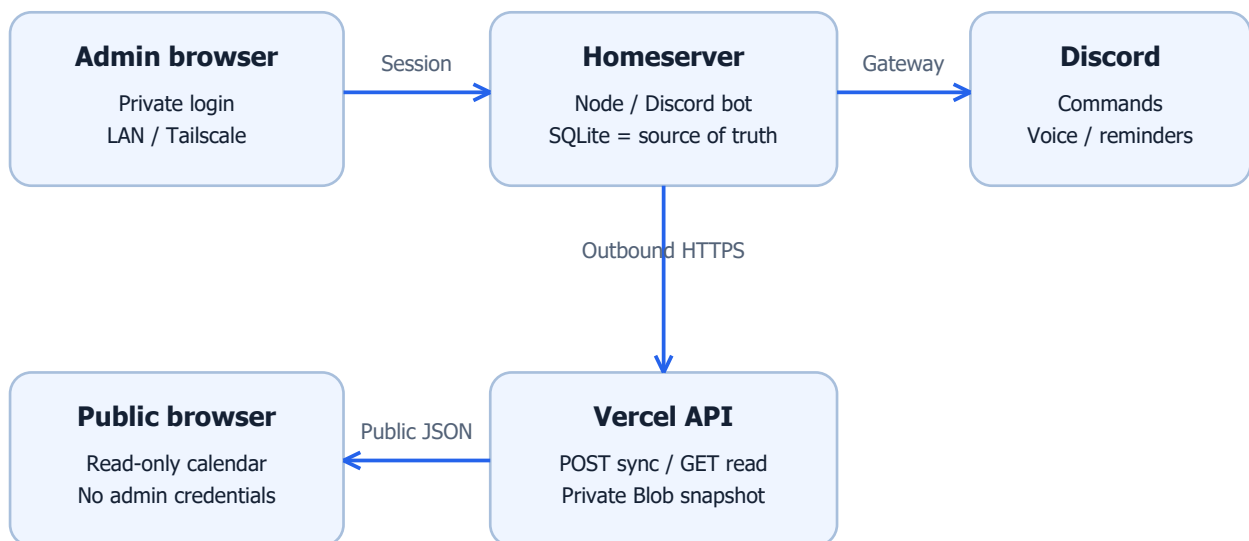
# TUTEL HUB

## Complete System Handbook

[Architecture](#) • [Code](#) • [Calendar](#) • [Bot](#) • [API](#) • [Operations](#)

English edition

by Double\_P | 2 October 2026 | Refactored source edition



# Contents

Select a chapter or page number to navigate. PDF outline bookmarks are included.

|                                                    |    |
|----------------------------------------------------|----|
| Contents                                           | 2  |
| 01 How to use this handbook                        | 6  |
| 02 Architecture and data ownership                 | 7  |
| 03 Technology choices and resource costs           | 9  |
| 04 Repository structure and module boundaries      | 10 |
| 05 Startup, concurrency and shutdown               | 11 |
| 06 Discord identity, installation and registration | 12 |
| 07 From a music request to audio                   | 13 |
| 08 Player state, queue progression and random mode | 14 |
| 09 Radio frequency, area and discovery             | 15 |
| 10 Calendar data and timezone semantics            | 16 |
| 11 Recurrence and holiday expansion                | 17 |
| 12 Reminder schedules and delivery guarantees      | 18 |
| 13 Discord embeds, buttons and deep links          | 19 |
| 14 SQLite explained and configured                 | 20 |
| 15 Tables, keys and persisted settings             | 21 |
| 16 Passwords and authenticated sessions            | 23 |
| 17 Authorization, CSRF and channel setup PIN       | 24 |
| 18 Express routes and browser security             | 25 |

19 Calendar UI, colors and mobile behavior

26

20 Shared assets, manifest and offline behavior

27

21 Public projection and snapshot generation

28

22 Outbound sender and retry state

29

23 Vercel receiver, schema and Blob concurrency

30

24 Public reads, cache layers and cost model

31

25 Environment variables and secret ownership

32

26 Local installation and development workflow

33

27 Homeserver rollout and Tailscale access

34

28 GitHub and Vercel releases

35

29 Backup, restore and release rollback

36

30 Troubleshooting and operational evidence

37

31 Maintenance, extension recipes and limits

38

Appendix A File-by-file code map

39

Appendix B HTTP contracts and access

48

POST /api/login . . . . .

48

GET /api/session . . . . .

48

POST /api/logout . . . . .

48

POST /api/password . . . . .

48

GET /api/bot . . . . .

48

POST /api/bot/toggle . . . . .

49

POST /api/control/music . . . . .

49

GET /api/calendar-sync . . . . . 49

POST /api/calendar-sync . . . . . 49

GET /api/guilds . . . . . 49

GET /api/guilds/:id/channels . . . . . 50

POST /api/settings/discord . . . . . 50

POST /api/settings/discord/test . . . . . 50

GET /api/events . . . . . 50

POST /api/events . . . . . 50

PUT /api/events/:id . . . . . 51

DELETE /api/events/:id . . . . . 51

GET /api/health . . . . . 51

GET /login . . . . . 51

GET /login.js . . . . . 51

GET /app.js . . . . . 51

GET /app.css . . . . . 52

GET / . . . . . 52

GET /admin . . . . . 52

GET /viewer . . . . . 52

GET /api/preferences . . . . . 52

PUT /api/preferences . . . . . 52

GET / HEAD /api/calendar (Vercel) . . . . . 53

POST /api/calendar/sync (Vercel) . . . . . 53

Event mutation body . . . . . 53

Appendix C Environment reference 54

|                                            |    |
|--------------------------------------------|----|
| Appendix D Command reference               | 57 |
| Appendix E Database schema reference       | 59 |
| users . . . . .                            | 59 |
| sessions . . . . .                         | 59 |
| events . . . . .                           | 59 |
| guild_config . . . . .                     | 59 |
| reminder_log . . . . .                     | 59 |
| app_settings . . . . .                     | 59 |
| calendar_deliveries . . . . .              | 59 |
| calendar_latest . . . . .                  | 60 |
| Appendix F Worked examples                 | 61 |
| All-day / 整天 . . . . .                     | 61 |
| Weekly / รายสัปดาห์ . . . . .              | 61 |
| Concurrent work / งานพร้อมกัน . . . . .    | 61 |
| Failed upload / ส่งไม่สำเร็จ . . . . .     | 61 |
| Appendix G Glossary                        | 62 |
| Appendix H References and further reading  | 65 |
| Appendix I Discord command input contracts | 66 |

# 01 How to use this handbook

## Scope and reading route

Tutel Hub combines a Discord music/radio bot, a private administration panel, a Thai calendar and a public read-only calendar. This handbook describes the code after the structural refactor dated 2 October 2026. It distinguishes implemented behavior from recommendations. Example events and credentials are fictional; no real secret or database content is reproduced.

Start with the architecture and startup chapters. Then follow an event through the calendar model, SQLite, snapshot generator and API. Study audio separately: its processes and failure modes are different from HTTP and calendar persistence. The appendices map every maintained code file, route and environment setting to its responsibility.

## What the guide does not assume

You do not need prior MySQL experience. A database is explained as tables and persistent records; an API as a contract between programs; a session as a temporary authenticated identity. Commands are accompanied by their purpose, so they are not merely copy-and-paste recipes.

External platforms can change. The lockfiles record the intended dependency versions, while official references at the end provide a route to current platform rules. Radio availability, upstream music access and government holiday declarations are not guaranteed by this repository. Deployment success and actual audible playback are different evidence.

## Learning safely

Experiment with a separate local database and no production Discord token or public-write secret. Follow one harmless event first. This lets you observe data flow without competing with the homeserver bot or publishing private experiments.

## 02 Architecture and data ownership

### Two runtimes

The homeserver runs a long-lived Node process. It owns SQLite, admin authentication, event editing, Discord connection, per-guild audio players, reminder scheduling and outbound publication. Vercel runs a separate public frontend and serverless API. Its private Blob stores a sanitized calendar snapshot. The homeserver database is authoritative; Blob is a public projection, not the master database.

### Direction of communication

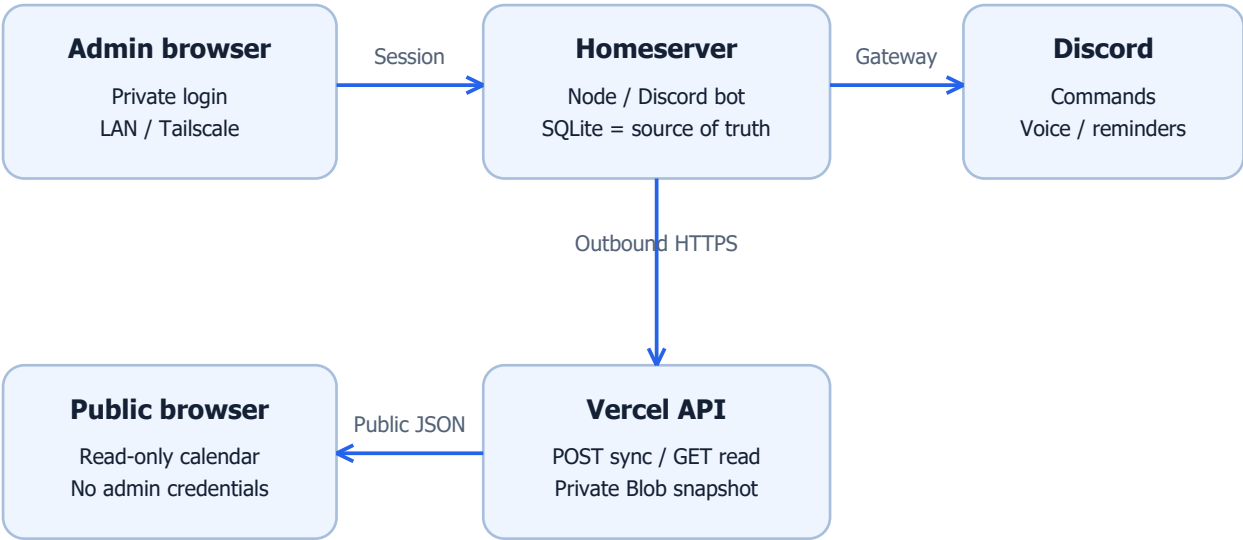
An admin browser sends authenticated requests to homeserver. The server writes SQLite and initiates HTTPS requests to Vercel. Public browsers read Vercel's API. Discord interactions arrive through the bot Gateway connection, while replies and messages use Discord APIs. These channels have separate identities and access rules.

A changing residential IP does not break the sender because it connects outward to a stable hostname. Private admin access still requires a reachable LAN address or Tailscale. Outbound data synchronization does not open the home web server to the public internet.

### Read-only is not confidential

Public visitors cannot edit through the public app. However, every exported event title and description is visible. There is currently no per-event visibility field. Even a guild-scoped custom event can appear in the public snapshot: guild scope controls Discord delivery, not public confidentiality. Never enter private details assuming that login on the admin side makes exported data private.

The architecture illustration shows boundaries rather than merely folder names: the important separation is between authoritative data, public projection, private administration and external media/Discord services.



Technical diagram drawn from project behavior; chapter text explains limitations.

## 03 Technology choices and resource costs

### Runtime and packages

JavaScript ES modules are enabled by type: module. Node 24 or newer is required by this repository; built-in node:sqlite provides persistent storage without a separate database daemon. Server JavaScript can use filesystem/process APIs. Browser JavaScript runs separately and cannot directly open the server database.

Discord.js models clients, interactions, channels and command definitions. @discordjs/voice owns voice connections and audio players; opusscript provides an Opus encoder. yt-dlp is an external executable for metadata and audio transport. FFmpeg decodes sources into PCM; ffmpeg-static supplies a binary unless FFMPEG\_PATH overrides it. Express handles HTTP routes, Helmet supplies security headers, dotenv loads server configuration and date-holidays contributes holiday definitions. The public Vercel package installs @vercel/blob independently of bot dependencies.

### Why this is lightweight

SQLite removes a MySQL service and its configuration overhead. Streaming avoids a persistent library of downloaded songs. Nevertheless, audio bytes still enter the machine and occupy pipes/buffers. FFmpeg decoding and Opus encoding consume CPU. Each simultaneously active guild can add audio subprocesses.

A Node heap limit does not cap total RAM: native buffers, yt-dlp and FFmpeg are outside that budget. A one-gigabyte server may suit modest use but has no unlimited capacity guarantee. Measure actual memory/CPU before promising concurrency. Professional structure makes these trade-offs explicit; it does not require introducing a framework or database server merely for appearance.

## 04 Repository structure and module boundaries

### Responsibility-based layout

src/index.js composes the runtime and owns shutdown. bot manages Discord lifecycle; commands separates definitions, music, radio and calendar handlers; music separates state, media streams and radio discovery. auth holds password/account/session logic, while database owns connection, schema and persisted settings.

calendar separates recurrence mathematics, event storage/query expansion, reminders, message construction, options, holiday sources, snapshots and publication. web/server.js assembles Express and registers focused route modules. web/middleware holds shared access checks. web/public is the authoritative location for shared browser assets.

vercel-public is an independently deployable public app. netlify-public retains the legacy provider frontend. scripts contains maintenance tools; deploy contains a service template; docs contains guides. private, data and backups remain outside version control.

### Stable facades

calendar/service.js and calendar/db.js preserve existing exported names while implementations move into focused modules. commands/handlers.js combines handler groups. A facade reduces interface churn and makes a refactor less disruptive. Keep it thin: it should not become a new monolith.

A module is defined by a coherent job, not an arbitrary line limit. The browser still uses classic scripts; converting everything to browser modules would alter loading and deployment and belongs in a separate change. The appended CSS override order is preserved rather than reordered blindly. Shared copies are generated deliberately, not independently edited. The file appendix explains where to make each kind of future change.

## 05 Startup, concurrency and shutdown

### Startup sequence

index.js loads dotenv/config. Importing the database connection creates its directory, opens DatabaseSync and initializes schema. initializeAccounts creates only missing accounts. The reminder scheduler and publisher start, HTTP begins listening, then bot login is attempted unless bot\_enabled is zero. Missing Discord credentials are caught and logged, allowing the panel to remain usable. A database-opening error happens earlier and can stop startup.

Importing the database facade is therefore not side-effect-free. A maintenance script that imports it can create/open a database even if it only intended to inspect a helper. Read the import graph before adding such scripts.

### Async versus synchronous work

Network requests and child-process pipes can progress while JavaScript waits. DatabaseSync and script calls are synchronous and briefly occupy the Node thread. Reminder and publisher busy flags prevent overlapping intervals within one process. They do not coordinate multiple production replicas.

### Graceful shutdown

SIGINT/SIGTERM trigger one guarded shutdown: stop scheduler/publisher work, stop players and Discord, wait for HTTP closure and exit. This avoids leaving audio subprocesses active. SQLite is not explicitly closed in this shutdown; process exit releases it. An explicit close could be a later lifecycle improvement rather than an unannounced behavior change.

Keep one production bot process. Starting the Windows clone with the same live token can create competing interaction handlers and reminder schedulers. A running control panel without a token is a valid development mode, not proof of a broken bot installation.

## 06 Discord identity, installation and registration

### Three separate actions

CLIENT\_ID identifies the application; DISCORD\_TOKEN authenticates the bot; GUILD\_ID optionally identifies a test server. Inviting installs the application in a guild. Logging in connects the bot. Registration publishes command definitions. None of these actions replaces the other two.

The invite uses bot and applications.commands scopes. Permissions include View Channels/Send Messages for text and Connect/Speak for voice. runtime uses Guilds and GuildVoiceStates intents, not message-content parsing for commands.

### Registration scopes

npm run register publishes globally. npm run register:guild targets the configured test guild. Both read commands/definitions.js and use REST. Guild/global definitions can coexist and look duplicated. Inspect scope before deleting; duplicate-looking entries do not automatically mean two bot processes. Registration does not fix media failures or slow interaction acknowledgment.

### Dispatch and acknowledgment

InteractionCreate routes buttons to handleCalendarButton and slash commands to commandHandlers[commandName]. Unknown commands are ignored. Errors are logged and the runtime attempts a private reply/follow-up. Slow work should use deferReply first, then editReply. Discord requires the initial response within three seconds; delay can invalidate an interaction.

Current families are music controls plus randommusic; radio list/play; calendar setup/add/list/delete/config/test. The implemented channel-setting subcommand is setup, not set. Definitions describe inputs and bounds, while handlers implement behavior. Changing only a handler cannot publish a new slash-command option.

## 07 From a music request to audio

### Metadata phase

/play validates guild and caller voice membership, defers the response and calls resolveTrack. URLs pass directly; names become search queries. With scsearch1 configured, search actually asks for ten SoundCloud results and selects an unknown-duration or longer-than-30-second track. This avoids some previews but cannot guarantee a full song.

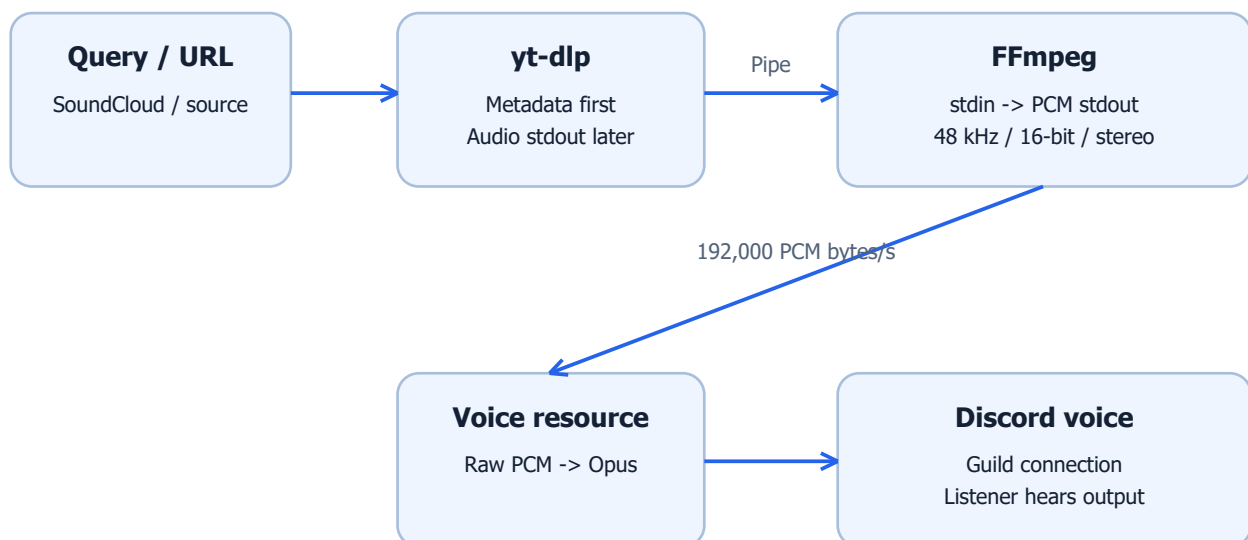
resolveQuery spawns yt-dlp for JSON. stdout collects metadata; stderr collects diagnostics. Nonzero exit rejects. Output exceeding two million characters triggers termination. The track object contains title, webpage URL and duration, not an audio file. Installing npm dependencies does not install yt-dlp.

### Streaming phase

yt-dlp writes media to stdout with -o -. Its pipe feeds FFmpeg stdin. FFmpeg emits signed 16-bit PCM at 48 kHz stereo. createAudioResource declares Raw input and the voice library encodes it to Opus for Discord. One second of this PCM is  $48,000 \times 2 \times 2 = 192,000$  bytes before compression.

No persistent song archive is created. Network data still reaches the machine and occupies memory/pipes: no saved file is more accurate than no download. Stop callbacks kill both subprocesses. FFmpeg ignores expected EPIPE input errors after closure and logs other diagnostics.

The PCM counter estimates seconds produced; it does not prove the listener heard sound. A voice connection can exist while the source is silent, the transcoder fails or permissions prevent output. Diagnose each stage rather than reinstalling everything at once.



Technical diagram drawn from project behavior; chapter text explains limitations.

## 08 Player state, queue progression and random mode

### State ownership

player.js keeps a Map keyed by guild ID. Each state owns queue, current track, audio player, connection, stream, randomMode, radio, randomHistory, loadingNext and generation. Music queues live in RAM and disappear on restart; the SQLite calendar does not persist them.

ensureConnection joins and subscribes the player. An existing connection in another channel is rejected instead of moving silently. NoSubscriberBehavior.Pause stops consumption without a subscriber. Disconnection destroys the state.

### Progression and race protection

playNext prevents overlapping loads and chooses a queued track or random search result. Generation checks stop old asynchronous results from restarting playback after stop/mode changes. On Idle, normal playback clears the current track and starts the next. Enqueue acknowledgment can therefore succeed before an audio failure occurs.

skip stops the current stream and allows Idle progression. stop increments generation, disables random/radio, clears queue/current and stops audio, retaining the connection. leave destroys the connection and removes the Map entry. pause/resume control the player.

Random mode searches one of several Thai music topics on SoundCloud and excludes recently used URLs. It is not Spotify or a curated recommendation API. Radio mode clears the queue and random mode. On radio Idle, a five-second retry checks that the same state/station remains before restarting. Mode transitions matter because timers and async searches can finish after a user has changed intent.

## 09 Radio frequency, area and discovery

### A frequency is not a stream

107.75 is an FM label, not an internet address. Different provinces reuse frequencies. The bot has no tuner; it needs an online HTTP/HLS stream. A station without a discoverable feed cannot be played merely by entering its frequency.

radio-directory.js combines a known-station catalog, regional aliases and Radio Browser's Thailand directory. The request times out after eight seconds and results are cached ten minutes. lastcheckok = 1 is a directory check result, not a current audio measurement.

### Matching

/radio list merges catalog matches with popular directory entries and limits the displayed set. /radio play first checks catalog aliases/name/frequency, then directory name/tags and area hints. Similar candidates can cause a request for a more precise name rather than an arbitrary choice. Metadata may be incomplete, and country/region coverage is not guaranteed.

FFmpeg opens the selected URL directly with reconnection options and converts to PCM. HTTP success can still mean silence, an advert or a wrong feed. Record selected name/URL, FFmpeg stderr, player status and whether other stations work. A dynamic directory avoids preparing every station manually, but does not manufacture nonexistent streams.

The original frequency parser's behavior is retained by this structural refactor. Parser correctness and live-audio acceptance are separate changes: a station lookup result must not be presented as proof of audible playback.

# 10 Calendar data and timezone semantics

## Base event versus occurrence

An events row stores the base definition: title/description, start/end, all-day/holiday flags, recurrence, reminders, mode, color, guild and creator. Query expansion adds occurrence\_at/occurrence\_end for each visible instance. A repeated event keeps the same ID, so public/frontend identity also uses occurrence time.

## Validation

saveEvent caps titles at 160 characters and descriptions at 2,000, accepts six-digit hexadecimal colors and defaults to #4285f4. Recurrence is none/daily/weekly/monthly/yearly. Minute offsets must be integers from zero to 10,080; at most five are accepted. Invalid dates/order/color throw validation errors.

All-day form endDate is inclusive, but storage end is exclusive next midnight. A one-day event on 13 October ends at the start of 14 October. Timed input without an end defaults to one hour. This convention makes interval overlap comparisons consistent.

## Thai time

localDateTimeToIso accepts Gregorian YYYY-MM-DD HH:mm in Bangkok and converts to UTC ISO. Helpers encode Thailand's UTC+7 assumption. UI may show Buddhist Era years; stored ISO remains Gregorian. 07:00 Thai time equals 00:00Z, not 07:00Z. Do not put a displayed Buddhist year into an ISO Gregorian date or subtract the offset twice.

A visitor abroad still sees the project's Thai timezone. Keeping storage as an instant and presentation as a chosen timezone avoids ambiguous strings that depend on whichever machine happens to read them.

# 11 Recurrence and holiday expansion

## Local calendar arithmetic

recurrence.js extracts Bangkok date parts. Daily/weekly advance one/seven days. Monthly advancement preserves the original anchor day and clamps shorter months: an event on the 31st can use February's last day and return to the 31st later. Yearly advancement handles month lengths too. It skips near the requested window rather than walking every historical occurrence, and limits expansion to 800 iterations.

Queries reject invalid/reversed ranges or windows over 400 days. An occurrence overlaps if its end is after the window start and its start is before the window end. Long events beginning before the visible period can therefore still appear.

## Generated system events

holidays.js combines date-holidays, explicit Thai important dates and holy-days.json. System holidays are generated for requested years rather than stored as custom event rows. IDs contain date and a title hash. They use standard reminders and category tagging. holy-days.json has a finite precomputed year range; it is not an eternal official calendar. Exceptional government declarations and lunar dates require maintained data.

Categories include public, substitute, bank, Buddhist, holy, royal, festival, Thai, international and custom. Some classification uses title patterns and an event can have several tags. Browser filters hide displayed items; reminder filters control server notifications. They do not delete events. This refactor preserves the existing holiday dataset rather than certifying completeness.

## 12 Reminder schedules and delivery guarantees

### Candidate times

Standard/all-day events default to noon on the previous day and 07:00 on the event day. Timed standard events use actual start for the day-of reminder. Guild options can change times or disable either schedule. Offset mode subtracts each minute offset from occurrence start.

The scheduler ticks immediately and every twenty seconds, expands eight days on either side of now, filters configured guild/channel/category settings and checks due schedules. A one-hour recovery window catches recent misses after restart but does not replay all old history. Busy prevents overlapping ticks in this process.

### Deduplication, not exactly once

`calendar_deliveries` uniquely identifies guild/channel/event/occurrence/schedule. `sendCalendarNotification` checks for a row, sends to Discord and stores message ID/payload. Legacy `reminder_log` checks some offset deliveries. Repeated normal ticks do not resend recorded deliveries.

If Discord accepts a message and the process crashes before recording it, a later tick can resend. Multiple processes can race between check and insert. One production process is the intended model.

### Changes after delivery

Updating an event clears legacy `reminder_log` but not all `calendar_deliveries`. Renaming an already delivered occurrence does not automatically resend its existing delivery key. Event deletion cascades event-linked legacy logs; textual delivery records may remain for message/button handling. These are current semantics, not a full audit/retention policy. Decide resend and cleanup behavior explicitly before implementing it.

# 13 Discord embeds, buttons and deep links

## Channel message

notifications.js constructs an embed using guild color, template and a by Double\_P footer. Placeholders are title/date/schedule/description/category. showDetails controls description insertion; template and final text are bounded. allowedMentions parse: [] avoids automatic mass pings from event text.

The Calendar button carries a stored delivery ID. calendar\_latest remembers the latest message per channel. Older active messages have components removed while their text remains. Deleted Discord messages are marked inactive.

## Ephemeral details

Clicking Calendar defers a private reply, loads the delivery payload and shows Open Calendar plus Dismiss. Dismiss includes the user's ID and rejects another user's custom ID. A thirty-second in-memory timer attempts to remove the private response. A process restart can cancel that timer; deletion is not durable background scheduling.

calendarUrl builds /calendar with date, event and at parameters. date selects the Thai day; event identifies the base event; at selects a repeated instance. The public client can navigate and open details. The link is public, not an admin login link. Configure the public URL, not a private LAN panel address.

Delivery payloads capture content at send time. An event edited later can differ from that historical snippet. Keep notification history and current calendar state conceptually separate.

# 14 SQLite explained and configured

## Embedded but persistent

SQLite runs inside the application and opens a local file. There is no MySQL service, database port or database account to create. Tables and SQL still exist; embedded does not mean temporary. DATABASE\_PATH selects the actual file. DATA\_DIR stores initial credential files and does not relocate the database. Relative paths depend on working directory; use repository-root startup or absolute server paths.

connection.js creates the directory and opens DatabaseSync. schema.js creates missing tables and checks event columns before adding reminder\_mode/color. This is additive compatibility logic, not a numbered general migration system. Editing CREATE TABLE does not change existing column types. Plan a migration and backup for structural changes.

## WAL and concurrency

WAL journal mode, synchronous NORMAL, foreign\_keys ON and busy\_timeout 5000 are configured. WAL permits convenient coexistence of readers and a writer; writes remain serialized and calls here are synchronous. It is not a replicated database cluster or a shared-network-file protocol.

The .sqlite file may have -wal/-shm sidecars. They are normal live files, not garbage to delete while running. Do not copy only the main file for an online backup. The application does not encrypt SQLite: password hashes are protected against direct password recovery, but event text/settings/delivery content remain readable to anyone with file access. Keep the database and backups private.

# 15 Tables, keys and persisted settings

## Identity

users holds unique username, password\_hash, role, must\_change and creation time. sessions holds token\_hash, user foreign key, csrf and expiry milliseconds. User deletion cascades sessions. Browser raw tokens are not stored as database keys.

## Calendar

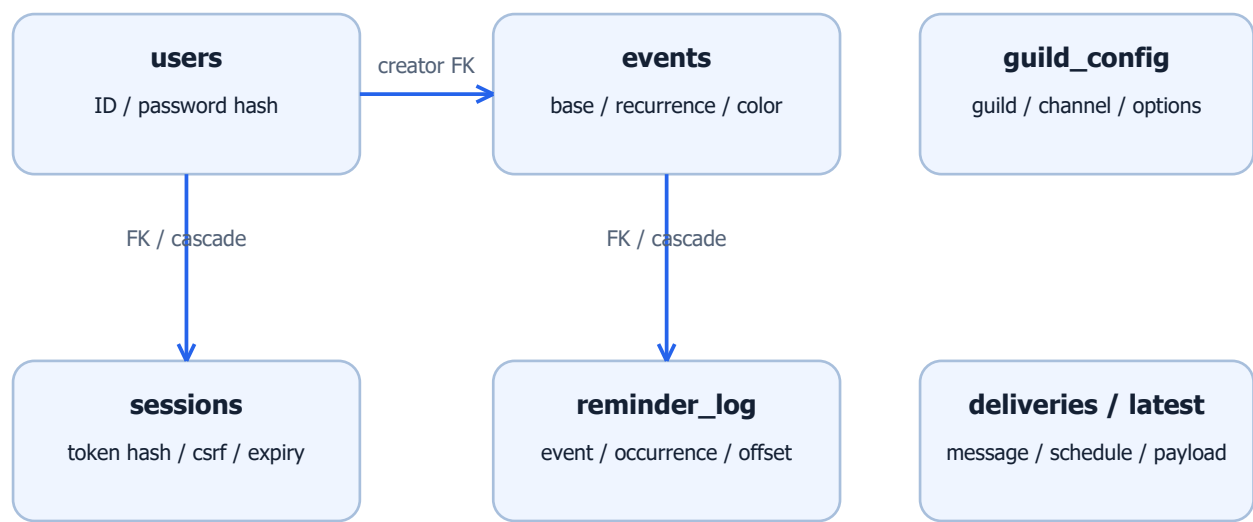
events stores base custom definitions; created\_by can reference a web user, while Discord creation uses null. guild\_id null is not restricted to one guild. guild\_config chooses channel/default offset per guild. app\_settings is string key/value storage for bot\_enabled, public URL, user preferences, guild reminder options and publisher state.

reminder\_log is an event-linked compound-key legacy log. calendar\_deliveries stores textual event identity, occurrence, schedule, channel, message ID, payload and active status. Its compound uniqueness supports send deduplication. calendar\_latest stores one latest message per channel.

## Queries and types

Indexes support starts\_at, guild and active channel-delivery lookups. Parameter binding keeps user values separate from SQL syntax. Settings are strings even when logically JSON, numbers or booleans. Consumers must parse and normalize. Missing settings return null; malformed preferences/options fall back.

Document each new setting's owner, default and parse behavior. Do not let the key/value table become an undocumented dumping ground. The relationship diagram shows which tables have foreign keys and which merely contain textual external IDs; those differences explain why some records cascade on deletion and others remain.



External Discord IDs are text; deliveries do not FK-link event IDs.

Technical diagram drawn from project behavior; chapter text explains limitations.

# 16 Passwords and authenticated sessions

## Bootstrap

Missing admin/viewer accounts are created, existing accounts preserved. A fresh admin can use ADMIN\_INITIAL\_PASSWORD of at least ten characters or a generated random password saved privately. must\_change requires the first password change. Bootstrap configuration does not reset an existing admin.

passwords.js uses scrypt, a random 16-byte salt, N 32768/r 8/p 1 and 64 derived bytes. The packed hash carries parameters/salt/key. Verification recalculates and uses a timing-safe comparison. Hashing is verification, not reversible encryption.

## Sessions and cookies

createSession generates a 32-byte random browser token plus a separate 24-byte CSRF token. SQLite stores SHA-256 of the browser token, not its raw value, with eight-hour expiry. The session cookie is HttpOnly, SameSite Strict, Path / and Secure when Express detects HTTPS. JavaScript obtains the CSRF value from the authenticated session endpoint.

Logout removes the session and clears its cookie. Password change invalidates the user's sessions and creates a replacement. Expired sessions are removed when presented; no global periodic session cleanup exists.

LAN HTTP and Tailscale HTTPS are different origins. Cookies and localStorage preferences do not automatically move between them. A successful login at one address does not imply login at another. Never add credentials to a public snapshot to make login seem easier; public viewing intentionally needs no admin account.

# 17 Authorization, CSRF and channel setup PIN

## Access pipeline

Protected reads use auth. Admin mutations use auth, admin, csrf: resolve session, verify role/password-change state, then validate x-csrf-token. Viewers cannot use admin control routes. A mustChange admin receives 428 until changing password.

CSRF protects against an unwanted request sent by an already authenticated browser. It is different from password guessing. SameSite Strict plus a session-specific header reduce this threat. The code uses timingSafeEqual; a future hardening should compare byte lengths safely before calling it, rather than character lengths for arbitrary Unicode input.

## Pre-login and rate limiting

The login page sets a short-lived tutel\_pre cookie whose value must be echoed in login JSON. Failed attempts are tracked by IP in a Map. Restart clears it and replicas do not share it; this is not MFA or durable attack auditing.

## PIN scope

CALENDAR\_SETUP\_PIN gates selecting/changing a Discord reminder channel. /calendar setup requires it; the web route checks it when channel changes. It is not required for normal event creation and is different from admin password and CALENDAR\_SYNC\_SECRET.

Existing Discord calendar add/delete/config handlers do not implement a separate admin-role policy. Delete is guild-scoped. PIN is not complete command authorization. Decide allowed roles as a separate feature instead of claiming that a protected channel setup protects every calendar operation.

# 18 Express routes and browser security

## Composition

server.js creates Express, trusts loopback proxies, disables x-powered-by, applies Helmet, limits JSON to 32 KB and sets no-store. It registers focused page/auth/event/preference/assets/sync/Discord/bot/health modules. Route paths and middleware order remain compatible after extraction.

Pages redirect missing sessions to login and roles to their own view. Static files are explicitly allowed rather than exposing the repository. The legacy publicEvent export remains, while real snapshot serialization belongs to calendar/public/snapshot.js.

## CSP and proxy behavior

Content Security Policy restricts scripts/styles/connections to same origin and blocks framing/objects. Adding CDN or inline code can fail under it. Solve a specific requirement instead of weakening all directives. Tailscale Serve provides HTTPS and proxies locally; trusted loopback forwarding lets Secure cookies behave correctly.

## Status and error meaning

Validation usually returns 400, missing session 401, forbidden role/CSRF 403, required password change 428, offline bot conditions 503 and public-sync failure 502. Route-local catches remain; this is not a complete centralized JSON error layer. Unhandled errors can reach Express defaults.

/api/health only reports HTTP availability. It does not certify Discord readiness, radio audio, database backup validity or last successful sync. The endpoint appendix describes actual auth and output, including legacy property names retained for client compatibility.

# 19 Calendar UI, colors and mobile behavior

## Browser roles

app.js owns authenticated event editing, notification settings and bot controls. calendar-view.js supplies shared month/week/day/agenda rendering. preferences.js owns theme/category preferences. Public calendar.js loads a read-only API snapshot. These are classic browser scripts, not Node modules.

All views display occurrences. Month cells stack event bars; timed views can position overlaps; selected-day details and event dialogs provide more context. Admin cell actions prefill a chosen date. Display filters do not alter stored events.

## Theme versus event data

Device theme is default until explicitly changed. app.css defines theme tokens, controls, dialogs and responsive overrides. Event color defaults blue and is stored per event. Discord embed color is a separate guild setting. Choosing blue for an event should not make the entire app blue.

The stylesheet has accumulated overrides. This refactor formats and centralizes it but preserves cascade order; blindly reordering selectors can break mobile/timed layouts. A separate visual redesign should review all views and breakpoints.

Touch navigation changes calendar periods. Desktop wheel navigation is scoped to calendar behavior; dialogs and scrolling lists must remain usable. Long web event text clips to available space without changing Discord ellipsis. Search/view/theme controls should remain aligned and large enough to tap. Saved preferences are origin-specific; public and admin addresses can therefore have different saved themes.

## 20 Shared assets, manifest and offline behavior

### Canonical assets

Edit `app.css`, `calendar-view.js`, `preferences.js`, `icon.png`, `app-icon.png` and `manifest.webmanifest` under `src/web/public`. `npm run assets:sync` copies these six files to `vercel-public/netlify-public` using a root resolved from the script URL. Generated copies remain in Git for independent deployment.

Provider-specific `index.html`, `calendar.js`, `sw.js`, API and config are not copied. This prevents the legacy static-JSON Netlify client overwriting the API-based Vercel client. Editing a generated copy alone will be overwritten next sync.

### Installable app

The manifest defines Tutel's installed name, icons and launch mode. `icon.png` is the original turtle; `app-icon.png` adds a designed background. `generate-app-icon.ps1` uses Windows System.Drawing and is maintenance tooling, not a server runtime requirement.

Admin workers cache only selected visual assets, not authenticated HTML/API. Public workers cache public assets and `/api/calendar` for offline fallback. Network-first reads favor fresh responses; activation removes old Tutel cache versions. Offline content is last-known, not current guaranteed data.

Remote service workers require a secure context; localhost has development exceptions. Installation behavior depends on browser/platform and is not guaranteed every visit. No persistent +App button is required. Changing cache versions helps stale assets retire, but does not replace careful data/cache reasoning.

## 21 Public projection and snapshot generation

### Field boundary

Internal events contain guild scope, creators and reminder settings. `publicEvent` in `calendar/public/snapshot.js` explicitly selects ID/title/description, all-day/holiday/publicHoliday, recurrence, base/occurrence dates, `systemHoliday`, categories and color. It excludes accounts, session hashes, channel configuration and delivery records. A field allowlist is not a privacy classifier: all expanded custom events are included.

### Time range and identity

`buildSnapshot` uses the current Bangkok year and expands previous/current/two following years. The end boundary is exclusive at the start of the next year. ID plus occurrence timestamp deduplicates, then entries sort by occurrence time. Output has `timezone/from/to/generatedAt/events`.

`exportSnapshot` writes `calendar.json` under `PUBLIC_SITE_DIR`. If meaningful content excluding `generatedAt` is unchanged, it keeps the existing snapshot/time. Changed content is written to `.tmp` and renamed, reducing partially written reads in the intended single-process workflow.

The staging JSON is ignored by Git and is not the live Vercel browser endpoint. Public clients read `/api/calendar`. Code releases and calendar data writes are independent. A generated four-year projection is useful for browsing; it is not a permanent archive of every past event.

When adding a new internal field, decide whether it belongs in this public allowlist. Omitting a field can preserve privacy; adding it should be a conscious contract change coordinated with receiver validation and browser rendering.

## 22 Outbound sender and retry state

### Scheduling

publish.js starts immediately and repeats every sixty seconds. Web event mutations request a five-second debounce to combine rapid edits. Discord mutations are caught by the periodic fallback. Busy permits one tick. Failure stores an error and delays retries exponentially from fifteen seconds to five minutes.

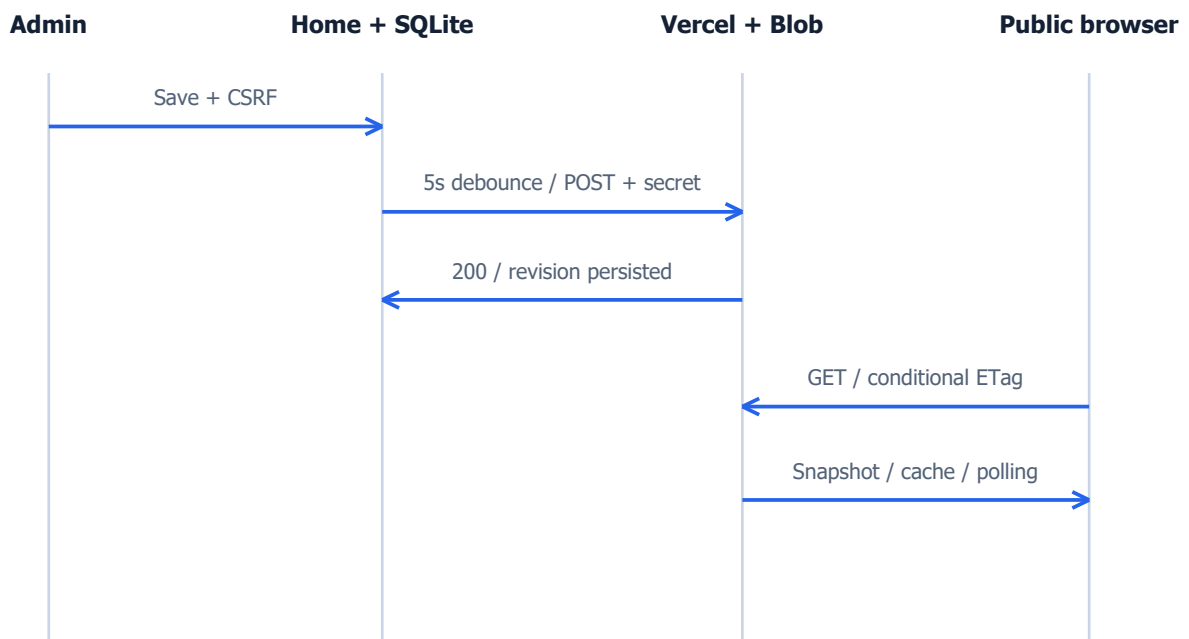
api-sync.js requires HTTPS without embedded URL credentials and a separate CALENDAR\_SYNC\_SECRET. It hashes timezone/range/events and compares successful hash plus endpoint. Normally unchanged data skips POST. Forced sync bypasses this optimization.

### Monotonic revisions

New content gets a revision greater than saved/pending revision and at least the current millisecond clock. Pending hash/revision is persisted before sending. Retrying the same content reuses it. The body adds schemaVersion 1 and generatedAt; headers use Bearer secret and application/json with thirty-second timeout.

Success updates successful hash/endpoint/revision/time and clears error. A 409 with the receiver revision advances pending revision for the next retry. Failure does not replace the last successful hash. If configuration is absent, pending is returned rather than inventing credentials.

The house initiates outbound HTTPS. NAT and changing home IP are not write-identity dependencies. During home outages the public calendar keeps the last snapshot; data synchronizes later. This is eventual consistency, not a distributed database transaction. The sequence diagram separates local commit, authenticated upload and later browser observation.



Technical diagram drawn from project behavior; chapter text explains limitations.

## 23 Vercel receiver, schema and Blob concurrency

### Authenticated write

POST /api/calendar/sync accepts POST only, checks secret/Blob configuration, compares hashed authorization using timingSafeEqual, requires JSON and caps serialized input at two million bytes. It validates and stores; statuses distinguish method/auth/content-type/size/shape/stale/storage failures.

validateSnapshot requires schemaVersion 1, positive safe revision, Asia/Bangkok, parseable generatedAt, date-range labels and at most 10,000 events. Each event has an ID, bounded nonempty title/string description, valid base/occurrence date strings and positive occurrence duration. Color/category shapes are checked. Unknown event fields are removed and contentHash recomputed.

Validation is structural: it does not ensure every occurrence lies inside from/to or every category is a known catalog ID. New integrations should not infer stricter checks than exist.

### Private Blob

store.js uses calendar/latest.json. Server-to-store authentication uses connected BLOB\_STORE\_ID/OIDC or a read/write token. The home's shared secret authenticates HTTP to Vercel, not Blob access directly. Public visitors do not get either credential.

saveSnapshot rejects older revisions. Equal revision/equal hash returns unchanged; equal revision/different hash conflicts. Overwrites use current Blob ETag with ifMatch and retry races up to three attempts. Initial writes disallow overwrite. This avoids stale concurrent replacement without requiring a database server. Blob remains object storage, not SQL tables or a distributed scheduler.

## 24 Public reads, cache layers and cost model

### Read contract

/api/calendar supports GET/HEAD. It reads private Blob server-side and serves sanitized JSON or headers. ETag uses contentHash; matching If-None-Match returns 304. Missing first snapshot and storage failures return 503. The private Blob URL is not the visitor-facing API.

### Freshness

Headers use browser max-age 0, CDN s-maxage 60 and stale-while-revalidate 60. The client polls each minute and on returning to the tab; a worker offers last-successful offline fallback. An edit passes local commit, debounce, upload, CDN revalidation and browser polling. Those stages mean immediate visibility is not guaranteed. Failures/background scheduling/backoff can extend the delay.

Distinguish generatedAt, server lastSync and browser fetch time. Inspect API revision when a page appears stale; repeatedly deploying source is not the correct response to every data-sync problem.

### Cost

Data writes no longer create website builds. They still consume function execution, Blob operations/storage and bandwidth. Hash skips, idempotency, CDN cache and conditional requests reduce work but do not make it free. A 304 can still cause origin access depending on cache.

A savings percentage requires measured visitors, edit rate, snapshot size and current plan pricing. The architectural benefit is separating source deployment from data synchronization, not a claimed fixed credit saving. Use Vercel usage measurements for financial decisions.

# 25 Environment variables and secret ownership

## Configuration groups

Discord: DISCORD\_TOKEN/CLIENT\_ID/GUILD\_ID. Media:

YT\_DLP\_PATH/FFMPEG\_PATH/MUSIC\_SEARCH\_PREFIX. Local runtime: CONTROL\_HOST/PORT, DATABASE\_PATH, DATA\_DIR. Bootstrap/access: ADMIN\_INITIAL\_PASSWORD and CALENDAR\_SETUP\_PIN. Public data: PUBLIC\_CALENDAR\_URL/SITE\_DIR/DEPLOY\_PROVIDER, CALENDAR\_SYNC\_URL/SECRET.

Production provider is calendar-api. VERCEL\_TOKEN/project/team identify legacy direct source-deployment tooling, not the data sender. Netlify values belong to the legacy provider. Vercel public runtime needs shared sync secret plus its connected Blob authentication, not Discord token.

Never put secrets in frontend JS/HTML/manifest, public JSON or client-exposed environment variables. The shared sync secret should be random and independent of admin password or short PIN. Changing only one side causes 401; both runtimes must use the matching new value.

## Safe editing

Copy .env.example only for a fresh clone. Do not overwrite an existing configured .env. Paths with spaces may be quoted; a path value identifies an executable, not an entire shell command. Windows and Linux media paths differ. Relative data paths depend on process working directory.

.gitignore avoids accidental tracking but cannot remove already committed secrets or prevent forced addition. Review staged files. Never print a whole production .env as a troubleshooting shortcut. The environment appendix documents each safe key without copying real values.

## 26 Local installation and development workflow

### Windows

Use Node 24+, npm and an accessible yt-dlp executable. In D:/tutel-bot, npm ci installs root dependencies from the lockfile. npm install is for deliberate dependency changes. Install/update yt-dlp separately. ffmpeg-static provides FFmpeg unless overridden.

For experiments select a separate DATABASE\_PATH/DATA\_DIR, leave live Discord credentials blank and do not enable production public writes. npm start opens the control panel and schedules background services. A fresh database writes private initial credentials; first login requires changing password. Missing Discord token is logged while the panel stays available. Guild/channel controls need a ready bot.

### Standalone public project

vercel-public owns package.json/lockfile for its API. Root installation does not install its Blob dependency; run npm ci inside that directory when needed. Opening index.html through file:// does not create /api/calendar. A server/hosting runtime is required for API behavior.

### Editing loop

Edit authoritative source, format it, run assets:sync for shared browser changes, inspect diff and update docs when contracts change. Formatting and syntax parsing are not acceptance tests. Do not log into the production bot merely to check indentation.

Examples in the command appendix explain where to run commands and what each changes. Keep an existing local database intact; choosing a new test path is safer than deleting the original to obtain a fresh install.

## 27 Homeserver rollout and Tailscale access

### Existing production paths

Application source and private .env live under /home/doublep/tutel-bot/app. SQLite is under /home/doublep/tutel-bot/data. A separate public staging folder holds generated snapshots. The active service is tutelbot.service; deploy/tutel-hub.service in the repo is a generic template that must match the actual working directory.

### Rollout

Back up source and take a consistent SQLite snapshot first. Transfer a candidate without env/private/data/node\_modules. Install from lockfile on the target. Review parsing/module paths, stop the active service, apply source, restart and observe logs. Keep the previous archive for rollback. Do not reset DB to solve an import error. Production npm installs omit formatting dev tooling, and external yt-dlp must still be accessible to the service.

CONTROL\_HOST selects bind address. LAN access uses the home address; loopback can sit behind a trusted proxy. Tailscale Serve supplies HTTPS for tailnet devices. Public visitors are not automatically tailnet members. The outbound sender does not require inbound port forwarding.

### DNS issues

If the tailnet hostname does not resolve, check client Tailscale connection/login/MagicDNS before changing HTML. Browser secure-DNS overrides can interfere with private names. HTTP LAN and HTTPS tailnet are different origins and sessions. Admin reachability and public read-only calendar reachability are separate tasks.

## 28 GitHub and Vercel releases

### Source deployment versus data sync

GitHub stores source, safe examples, locks and docs. A connected push can trigger Vercel builds. Event updates write Blob through API without Git. Receiver code changes still require a source release.

The local public folder is vercel-public. Vercel Root Directory must match after rename; framework Other, install npm ci --omit=dev, output '.', functions under api. Deploying the repo root incorrectly mixes bot/admin materials with public hosting. Local folder rename does not update platform settings.

vercel.json's ignoreCommand recognizes [skip vercel] in a commit message. Such commits can update GitHub without a build. A normal release should omit that marker. Preserve environment/store connections while changing the root.

### Git sequence

status and diff inspect; add selects; diff --cached --name-only lists the exact staged files; commit records; push sends commits. Staging is not upload/deploy. A clear message is refactor: organize application modules and add bilingual handbooks. Review staged files for secrets and data, not just filename count.

Repository-not-found can indicate URL or permissions. HTTPS Git uses supported credentials, not ordinary password authentication. Avoid inserting tokens into remote URLs. A completed push and a READY deployment prove source delivery, not correct audible playback or fresh calendar data.

## 29 Backup, restore and release rollback

### Consistent backup

npm run db:backup checks source existence and new destination, then uses SQLite VACUUM INTO with a busy timeout. This creates a consistent database snapshot rather than copying only a main file during WAL activity. Names use filesystem-safe timestamps. Keep env backups separately and securely: DB contains accounts/events/settings, not environment tokens or executables.

### Restore

Stop every writer. Preserve current database and paired sidecars in a separate recovery folder. Put the selected consistent backup at DATABASE\_PATH without stale WAL/SHM. Confirm permissions, start and inspect. Never restore over a live file.

An older backup restores older passwords/sessions/configuration/delivery history. Deduplication and public/home revision differences can therefore change. Decide whether to force-sync recovered state. A restart alone does not resolve every data conflict. Keep a second backup location if home disk loss matters.

### Code rollback

Stop service, restore previous source/lock, install matching dependencies if needed, restart. Additive schema changes are generally friendlier than destructive migrations, but future schema changes need compatibility planning. Current refactor preserves table meaning. Frontend rollback can require a fresh asset load/cache-version update.

Backup creation is not the same as a tested recovery procedure. Verify recoverability on an isolated path when doing an explicitly planned recovery exercise, never by overwriting production just to check the file.

## 30 Troubleshooting and operational evidence

### Follow the failing layer

Startup failure: inspect Node version, imports/dependencies, DB permissions and bind address. HTTP works but bot offline: inspect token/bot logs. Commands absent: inspect application identity, registration scope and invite. Music acknowledged but silent: inspect search, yt-dlp exit, FFmpeg PCM, player and voice permissions. A missing Opus module is an installation issue; a Windows Application Control block is an OS-policy issue. Do not disable security policy blindly.

Channel loading needs a ready bot and correct permissions. Admin 401 means session/login; 403 may mean role/CSRF/PIN; 428 means password change. Public write 401 means credentials disagree; 409 means revision conflict; 503 can mean configuration, first-sync waiting or storage failure.

For stale public pages compare admin data, lastSync/error, API revision and browser network/cache. Source deployment does not guarantee data sync. For radio record selected URL/station and FFmpeg output; directory lastcheckok does not prove sound.

### What evidence proves

Ready proves Discord login. /api/health proves HTTP response. READY proves deployment finished. None proves audible music or every reminder delivered. PCM activity proves decoded bytes, not that a listener heard them. End-to-end listening and actual notification checks require distinct acceptance work.

Redact cookies, tokens, Authorization and private env values from diagnostics. Record timestamps and IDs instead. Make a change at the failing layer rather than introducing unrelated code churn.

# 31 Maintenance, extension recipes and limits

## Style and tooling

.editorconfig sets UTF-8/LF/two spaces/final newline. Prettier sets width 100, semicolons, single quotes and trailing commas. format writes; format:check reports. Private/generated large data and lockfiles are excluded appropriately. Use responsibility names and explain non-obvious decisions, not every obvious statement.

Add a command by updating definitions, implementing its handler group, deciding permissions/acknowledgment and registering after deployment. Add an event field across schema, save validation, expansion, optional public projection/receiver validation and browser/docs. Changing only a form can create a field that never persists. Add routes with explicit auth/role/CSRF/status contracts and avoid importing the top-level server back into services.

## Current limits

Queues/timers are in memory; reminders are best-effort deduplicated; holidays need maintenance; radio needs online feeds; all exported events are public; SQLite is single-host; Discord calendar authorization is limited; route-local error handling and CSRF byte-length handling can improve. CSS override order remains; asset changes require sync. No full CI/browser/audio suite is added merely by refactoring.

Monitor Node/FFmpeg/yt-dlp memory separately, update media extractors intentionally, curate broken streams and future holidays, keep backups/log retention. Future visibility flags, role controls, numbered migrations, isolated regression coverage and sync-age alerts are recommendations, not implemented features. Choose them with explicit requirements rather than overcomplicating a small installation.

# Appendix A File-by-file code map

Each entry identifies responsibility and declared function names. Read it with the behavioral chapters; names alone are not the implementation contract. Images are assets, and private runtime content is excluded.

## **.editorconfig**

Editor indentation, encoding and line ending conventions.

## **.env.example**

Safe empty credential template plus example service/media/sync settings.

## **.gitignore**

Excludes credentials, databases, backups, generated snapshots and caches.

## **.prettierignore**

Formatting exclusions for private/runtime/generated material.

## **.prettierrc.json**

Shared automatic formatting rules.

## **compose.yaml**

Optional constrained container with private data/public staging mounts and loopback port.

## **deploy/tutel-hub.service**

Generic user-service template; adapt WorkingDirectory to deployment layout.

## **Dockerfile**

Optional Node/system-FFmpeg/Python-yt-dlp container running as nonroot.

## **netlify-public/\_headers**

Provider hosting/cache/routing or upload exclusion rules.

## **netlify-public/\_redirects**

Provider hosting/cache/routing or upload exclusion rules.

## **netlify-public/app.css**

Generated shared asset; edit src/web/public and run assets:sync.

## **netlify-public/calendar-view.js**

Generated shared asset; edit src/web/public and run assets:sync.

Declared functions: escape, parts, key, date, startWeek, monthFirst, monthShift, format, overlaps, color, foreground, time

## **netlify-public/calendar.js**

Legacy read-only static calendar.json client, not API sync receiver.

Declared functions: toast, occursOn, showDetails, selectDay, renderList, renderCalendar, changeMonth, loadSnapshot, dateKey, today, validDay, esc

## **netlify-public/index.html**

Public/provider support asset; see corresponding hosting/frontend chapter.

## **netlify-public/manifest.webmanifest**

Generated shared asset; edit src/web/public and run assets:sync.

## **netlify-public/preferences.js**

Generated shared asset; edit src/web/public and run assets:sync.

Declared functions: theme, persist, hideInstall, deferInstall, showInstall, refresh, safe

## **netlify-public/sw.js**

Legacy public cache for the static snapshot provider.

## **package-lock.json**

Exact root dependency graph for reproducible npm ci.

## **package.json**

Root commands, engine requirement and bot/admin runtime/dev dependency declarations.

## **scripts/backup-db.js**

Consistent VACUUM INTO backup to a new private destination.

## **scripts/generate-app-icon.ps1**

Windows drawing utility for the background of installed-app turtle icon.

## **scripts/sync-public-assets.js**

Copies six canonical browser assets to standalone provider directories.

## **src/auth/accounts.js**

Boostraps missing accounts, looks up users and changes passwords.

Declared functions: initializeAccounts, userByName, changePassword

## **src/auth/passwords.js**

Encodes salted scrypt hashes and timing-safe verification.

Declared functions: decode, encode, hashPassword, verifyPassword

## **src/auth/sessions.js**

Creates hashed-token sessions, resolves expiry and deletes sessions.

Declared functions: createSession, getSession, deleteSession

## **src/bot/runtime.js**

Owns the singleton Discord client, startup promise, interaction dispatch and stop.

Declared functions: getDiscordClient, botStatus, startBot, stopBot

## **src/calendar/api-sync.js**

Hashes changed data, persists revision state and POSTs authenticated snapshots.

Declared functions: syncCalendarApi

## **src/calendar/categories.js**

Defines category vocabulary and title-based system classification.

Declared functions: eventCategories

## **src/calendar/db.js**

Compatibility exports for database/auth/settings callers.

## **src/calendar/events.js**

Validates and persists custom events; merges generated holidays and guild config.

Declared functions: listExpandedEvents, saveEvent, deleteEvent, saveGuildConfig, listGuildConfigs

## **src/calendar/holidays.js**

Combines holiday library, fixed/derived important dates and lunar data.

Declared functions: thaiImportantDays

## **src/calendar/holy-days.json**

Finite precomputed lunar/holy-day dataset; maintain its year coverage.

## **src/calendar/notifications.js**

Sends embeds, records deliveries and manages latest/private buttons.

Declared functions: `calendarUrl`, `removeOldCalendarButtons`, `sendCalendarNotification`, `handleCalendarButton`

## **src/calendar/options.js**

Normalizes notification categories/times/colors/templates and filters events.

Declared functions: `normalizeOptions`, `reminderOptions`, `saveReminderOptions`, `shouldNotify`, `notificationText`

## **src/calendar/public/archive.js**

Builds allowlisted stored ZIP entries and CRC32 for legacy Netlify.

Declared functions: `publicZip`, `crc32`

## **src/calendar/public/snapshot.js**

Defines public field projection, four-year expansion and atomic JSON export.

Declared functions: `publicEvent`, `buildSnapshot`, `exportSnapshot`

## **src/calendar/publish.js**

Orchestrates snapshot export, provider selection, debounce and retries.

Declared functions: `publishSnapshot`, `publicSyncStatus`, `requestCalendarSync`, `syncCalendarNow`, `startCalendarPublisher`, `syncPrefix`, `tick`

## **src/calendar/recurrence.js**

Converts Thai local time and expands anchored recurring occurrences.

Declared functions: `localDateTimeToIso`, `addLocal`, `fromLocal`, `eventOccurrences`, `formatThai`, `dateParts`

## **src/calendar/reminders.js**

Builds due schedules and runs nonoverlapping twenty-second ticks.

Declared functions: `reminderSchedules`, `runReminderTick`, `startReminderScheduler`, `tick`

## **src/calendar/service.js**

Compatibility exports for events, recurrence and reminders.

## **src/calendar/setup-pin.js**

Hashes PIN comparison and throttles failed attempts per caller key.

Declared functions: `checkCalendarSetupPin`, `hash`

## **src/calendar/vercel-publish.js**

Legacy deployment adapter; not the current API data receiver deployment path.

Declared functions: publishVercel

### **src/commands/calendar.js**

Implements setup/add/list/delete/config/test with guild scope.

Declared functions: calendar

### **src/commands/definitions.js**

Defines slash names, subcommands, options and input bounds.

### **src/commands/handlers.js**

Combines command families through a stable dispatch facade.

### **src/commands/music.js**

Resolves/enqueues tracks and exposes playback/queue/random controls.

Declared functions: play, randommusic, queue, skip, stop, pause, resume, nowplaying, leave

### **src/commands/radio.js**

Matches station/area, shows candidates and switches the player to radio.

Declared functions: radio

### **src/commands/shared.js**

Provides guild-only validation and duration display helpers.

Declared functions: duration, guildOnly

### **src/database/connection.js**

Creates directory, opens DatabaseSync and invokes schema initialization.

### **src/database/schema.js**

Owens SQL tables/indexes, PRAGMAs and additive event-column upgrades.

Declared functions: initializeSchema

### **src/database/settings.js**

Reads/upserts string values under named application setting keys.

Declared functions: setting, setSetting

### **src/deploy-commands.js**

Validates registration credentials and sends global or guild command definitions.

## src/index.js

Composes accounts, schedulers, HTTP and optional bot; handles signals.

Declared functions: shutdown

## src/music/player.js

Owns per-guild in-memory state, queue transitions, modes and cleanup.

Declared functions: getState, ensureConnection, playNext, enqueue, enableRandomMode, playRadio, getPlayer, pausePlayer, resumePlayer, skip, stop, destroyPlayer

## src/music/radio-directory.js

Provides catalog, regional aliases and cached Radio Browser queries.

Declared functions: getRadioDirectory, stationMatchesRegion, stationFrequency, norm

## src/music/stream.js

Spawns yt-dlp/FFmpeg, parses metadata and creates raw resources.

Declared functions: resolveQuery, resolveTrack, resolveRandomTrack, createTrackResource, createRadioResource

## src/web/channels.js

Checks sendability/permissions and produces permitted channel choices.

Declared functions: canSendReminders, sendableChannels

## src/web/middleware/security.js

Shared cookies, session authentication, admin checks and CSRF.

Declared functions: cookies, clearCookie, auth, admin, csrf, setSessionCookie

## src/web/public/app.css

Canonical theme/layout/component styles; existing cascade order is preserved.

## src/web/public/app.html

Authenticated panel markup, calendar, dialogs and setting controls.

## src/web/public/app.js

Admin UI state, HTTP wrapper, forms, settings and bot actions.

Declared functions: esc, api, toast, dayKey, fromIso, startOfWeek, init, forcePasswordChange, wire, showPage, loadEvents, renderCalendar, selectDay, renderEventList, openEvent, bkkInput, editEvent, saveEventForm, deleteCurrentEvent, loadGuilds, loadSettings, loadChannels, saveSettings, loadBot, musicAction, toggleBot, field, occursOn, changeMonth, syncAllDay, showDetails, populateReminderOptions, readReminderOptions, previewNotification, updateChannelPin, syncEventColor, loadPublicSync

## **src/web/public/calendar-view.js**

Shared period navigation, month/time/agenda rendering and gesture/layout behavior.

Declared functions: escape, parts, key, date, startWeek, monthFirst, monthShift, format, overlaps, color, foreground, time

## **src/web/public/login.html**

Login document and accessible form/branding markup.

## **src/web/public/login.js**

Submits credentials with pre-login token and follows role redirect.

Declared functions: cookie, loadToken

## **src/web/public/manifest.webmanifest**

Installed-app identity, launch presentation and turtle icons.

## **src/web/public/preferences.js**

Shared saved theme/filter controls and install/launch-related browser behavior.

Declared functions: theme, persist, hideInstall, deferInstall, showInstall, refresh, safe

## **src/web/public/sw.js**

Admin visual-only network-first cache; excludes private HTML/API.

## **src/web/routes/assets.js**

Explicit public asset allowlist; does not expose arbitrary source files.

Declared functions: registerAssetsRoutes

## **src/web/routes/authentication.js**

Login throttling, sessions, logout and password replacement.

Declared functions: registerAuthenticationRoutes

## **src/web/routes/bot.js**

Bot status/toggle and guild playback controls.

Declared functions: registerBotRoutes

## **src/web/routes/calendar-sync.js**

Reports publisher state and requests a forced sync.

Declared functions: registerCalendarSyncRoutes

## **src/web/routes/discord.js**

Guild/channel discovery, guarded channel configuration and actual test send.

Declared functions: registerDiscordRoutes

## **src/web/routes/events.js**

Authenticated event reads and admin mutations with sync requests.

Declared functions: registerEventsRoutes

## **src/web/routes/health.js**

Unauthenticated basic HTTP liveness response.

Declared functions: registerHealthRoutes

## **src/web/routes/pages.js**

Login pre-token and role-based page redirects/static entry scripts.

Declared functions: registerPagesRoutes

## **src/web/routes/preferences.js**

Persists per-user theme/filter preferences after normalization.

Declared functions: registerPreferencesRoutes

## **src/web/server.js**

Constructs secured Express and registers route families/listening lifecycle.

Declared functions: publicEvent, startControlServer

## **vercel-public/.env.example**

Safe environment template; fill secrets on the server, never publish values.

## **vercel-public/.vercelignore**

Provider hosting/cache/routing or upload exclusion rules.

## **vercel-public/api/calendar/sync.js**

Authenticated JSON write endpoint with method/type/size/shape validation.

Declared functions: handler, digest

## **vercel-public/api/calendar.js**

Public GET/HEAD, ETag/304 and CDN caching over private storage.

Declared functions: handler

## **vercel-public/app.css**

Generated shared asset; edit src/web/public and run assets:sync.

## **vercel-public/calendar-view.js**

Generated shared asset; edit src/web/public and run assets:sync.

Declared functions: escape, parts, key, date, startWeek, monthFirst, monthShift, format, overlaps, color, foreground, time

## **vercel-public/calendar.js**

Read-only client polling, details/day selection and deep links.

Declared functions: toast, occursOn, showDetails, selectDay, renderList, renderCalendar, changeMonth, loadSnapshot, dateKey, today, validDay, esc

## **vercel-public/index.html**

Public-only calendar shell and dialogs; no admin editing.

## **vercel-public/lib/store.js**

Blob I/O, allowlist validation, content hashes and conditional revision writes.

Declared functions: readSnapshot, validateSnapshot, saveSnapshot

## **vercel-public/manifest.webmanifest**

Generated shared asset; edit src/web/public and run assets:sync.

## **vercel-public/package-lock.json**

Exact standalone dependency versions; use npm ci in this folder.

## **vercel-public/package.json**

Standalone public dependency/scripts declaration; independent of root package.

## **vercel-public/preferences.js**

Generated shared asset; edit src/web/public and run assets:sync.

Declared functions: theme, persist, hideInstall, deferInstall, showInstall, refresh, safe

## **vercel-public/sw.js**

Public network-first/offline assets and API snapshot cache.

## **vercel-public/vercel.json**

Hosting headers, functions, calendar rewrite and skip-commit rule.

## Appendix B HTTP contracts and access

### POST /api/login

Access: no session required

Login throttling, sessions, logout and password replacement.

Implementation: `src/web/routes/authentication.js`

Body `username/password/csrf`. Echo the pre-login cookie; failures return 401/403 or throttled 429. Success sets an eight-hour cookie and returns `role`, `mustChange` and `redirect`.

### GET /api/session

Access: session

Login throttling, sessions, logout and password replacement.

Implementation: `src/web/routes/authentication.js`

Returns authenticated user `ID/username/role/mustChange` and CSRF. Use this CSRF for subsequent mutations; never send it publicly.

### POST /api/logout

Access: session + CSRF

Login throttling, sessions, logout and password replacement.

Implementation: `src/web/routes/authentication.js`

Requires the session CSRF header; deletes current hashed session and clears cookie. Returns `ok`.

### POST /api/password

Access: session + CSRF

Login throttling, sessions, logout and password replacement.

Implementation: `src/web/routes/authentication.js`

Body `currentPassword/newPassword`; verifies current password, requires at least ten new characters, invalidates old sessions and returns a new CSRF. Viewer and admin can change their own password.

### GET /api/bot

Access: admin + session

Bot `status/toggle` and guild playback controls.

Implementation: `src/web/routes/bot.js`

Returns `status/configured`, guild list and music summaries: playing, paused, queue length and voice channel. Queue details are in RAM.

## POST /api/bot/toggle

Access: admin + session + CSRF

Bot status/toggle and guild playback controls.

Implementation: `src/web/routes/bot.js`

Body enabled boolean-like value. Starts/stops Discord and persists bot\_enabled on success; returns status. Stop destroys players; web/calendar keep running.

## POST /api/control/music

Access: admin + session + CSRF

Bot status/toggle and guild playback controls.

Implementation: `src/web/routes/bot.js`

Body guildId/action: pause/resume/skip/stop/leave. Missing player 404; unsupported action or impossible skip 400. Does not enqueue new tracks.

## GET /api/calendar-sync

Access: admin + session

Reports publisher state and requests a forced sync.

Implementation: `src/web/routes/calendar-sync.js`

Returns provider, configured flag, lastSync and persisted error. It reports sender status, not browser freshness.

## POST /api/calendar-sync

Access: admin + session + CSRF

Reports publisher state and requests a forced sync.

Implementation: `src/web/routes/calendar-sync.js`

No event body required. Forces current publisher/export path and returns result plus status; errors return 502 with status. Force can upload unchanged content.

## GET /api/guilds

Access: admin + session

Guild/channel discovery, guarded channel configuration and actual test send.

Implementation: `src/web/routes/discord.js`

Ready bot required (otherwise 503). Returns guild IDs/names, channel configs with reminder options, categories, publicCalendarUrl and legacy netlify property containing current provider status.

## GET /api/guilds/:id/channels

Access: admin + session

Guild/channel discovery, guarded channel configuration and actual test send.

Implementation: `src/web/routes/discord.js`

Requires cached guild; fetches permitted sendable channels. Missing guild 404; channel/permission loading failure 500.

## POST /api/settings/discord

Access: admin + session + CSRF

Guild/channel discovery, guarded channel configuration and actual test send.

Implementation: `src/web/routes/discord.js`

Body `guildId/channelId/defaultReminder/publicCalendarUrl/options` and `secretPin` when changing channel. Checks ready bot, membership and View/Send permissions; stores URL origin, config and normalized options. Validation returns 400; PIN rejection 403.

## POST /api/settings/discord/test

Access: admin + session + CSRF

Guild/channel discovery, guarded channel configuration and actual test send.

Implementation: `src/web/routes/discord.js`

Body `guildId`. Requires stored channel and ready bot, selects first upcoming event within thirty days and sends a real embed with test schedule key. This is an external side effect, not a dry run.

## GET /api/events

Access: session

Authenticated event reads and admin mutations with sync requests.

Implementation: `src/web/routes/events.js`

Query `from/to` are parseable dates defining a positive window  $\leq 400$  days; optional `guild` includes global and that guild events. Returns expanded custom + system events, including admin-side fields.

## POST /api/events

Access: admin + session + CSRF

Authenticated event reads and admin mutations with sync requests.

Implementation: `src/web/routes/events.js`

Body event fields documented below; strips any supplied ID and `secretPin`, saves under web user ID and requests debounce sync. Returns new ID.

## PUT /api/events/:id

Access: admin + session + CSRF

Authenticated event reads and admin mutations with sync requests.

Implementation: `src/web/routes/events.js`

Path ID overrides body ID. Same save validation; missing event returns 400 in this route. Requests sync and returns ID. No separate per-event creator ownership rule is implemented for admins.

## DELETE /api/events/:id

Access: admin + session + CSRF

Authenticated event reads and admin mutations with sync requests.

Implementation: `src/web/routes/events.js`

Deletes chosen event as admin, requests sync, returns ok; missing returns 404. Does not delete already sent Discord text.

## GET /api/health

Access: no session required

Unauthenticated basic HTTP liveness response.

Implementation: `src/web/routes/health.js`

Returns `{ok:true}` with no login. HTTP liveness only.

## GET /login

Access: no session required

Login pre-token and role-based page redirects/static entry scripts.

Implementation: `src/web/routes/pages.js`

Sets `tutel_pre` if absent (15 minutes), then serves `login.html`. No account state is returned.

## GET /login.js

Access: no session required

Login pre-token and role-based page redirects/static entry scripts.

Implementation: `src/web/routes/pages.js`

## GET /app.js

Access: no session required

Login pre-token and role-based page redirects/static entry scripts.

Implementation: `src/web/routes/pages.js`

## GET /app.css

Access: no session required

Login pre-token and role-based page redirects/static entry scripts.

Implementation: `src/web/routes/pages.js`

## GET /

Access: session determines redirect; anonymous -> login

Login pre-token and role-based page redirects/static entry scripts.

Implementation: `src/web/routes/pages.js`

## GET /admin

Access: session + matching role (checked inside page handler)

Login pre-token and role-based page redirects/static entry scripts.

Implementation: `src/web/routes/pages.js`

## GET /viewer

Access: session + matching role (checked inside page handler)

Login pre-token and role-based page redirects/static entry scripts.

Implementation: `src/web/routes/pages.js`

## GET /api/preferences

Access: session

Persists per-user theme/filter preferences after normalization.

Implementation: `src/web/routes/preferences.js`

Reads preferences: <user ID> from settings and parses JSON; malformed/absent data becomes an empty object.

## PUT /api/preferences

Access: session + CSRF

Persists per-user theme/filter preferences after normalization.

Implementation: `src/web/routes/preferences.js`

Body theme and filters. Theme allowed system/light/dark; filters are known category IDs. Stores normalized JSON and returns ok.

## GET / HEAD /api/calendar (Vercel)

Public snapshot read without login. Supports ETag/304; waiting/storage failure returns 503.

## POST /api/calendar/sync (Vercel)

Bearer CALENDAR\_SYNC\_SECRET plus JSON schemaVersion/revision/timezone/generatedAt/from/to/events. Two million bytes/10,000 events. Success 200, conflict 409, unauthorized 401, method 405, type 415, size 413, validation 400, storage 503.

## Event mutation body

| Field                     | Meaning / ความหมาย                                                 |
|---------------------------|--------------------------------------------------------------------|
| title / description       | Text bounds 160 / 2000; public when exported.                      |
| date / endDate / allDay   | All-day dates, inclusive form end converted to exclusive midnight. |
| startsAt / endsAt         | Thai local YYYY-MM-DD HH:mm; timed event inputs.                   |
| recurrence                | none, daily, weekly, monthly, yearly                               |
| reminders / reminderMode  | Minute offsets, or standard previous-day/day-of schedules.         |
| guildId / holiday / color | Guild scope, holiday flag, #RRGGBB default #4285f4.                |

# Appendix C Environment reference

## DISCORD\_TOKEN

Bot authentication secret; homeserver only.

## CLIENT\_ID

Application ID used for command registration.

## GUILD\_ID

Optional test-server ID for guild registration.

## YT\_DLP\_PATH

Executable name/full path, not an entire command.

## FFMPEG\_PATH

Optional override; blank selects ffmpeg-static.

## MUSIC\_SEARCH\_PREFIX

Name-search provider; example scsearch1.

## CONTROL\_HOST

Bind interface; choose loopback/LAN deliberately.

## CONTROL\_PORT

HTTP listening port; default 3000.

## DATABASE\_PATH

Actual SQLite file, relative to working directory if not absolute.

## DATA\_DIR

Initial credential-file directory, not an automatic DB relocation.

## ADMIN\_INITIAL\_PASSWORD

Fresh admin only; blank generates random, existing accounts unchanged.

## CALENDAR\_SETUP\_PIN

Channel selection/change guard, separate from login and API secret.

## **PUBLIC\_CALENDAR\_URL**

Public deep-link origin, not admin URL.

## **PUBLIC\_SITE\_DIR**

Private/local snapshot staging folder.

## **PUBLIC\_DEPLOY\_PROVIDER**

calendar-api for current data sync; others are legacy adapters.

## **CALENDAR\_SYNC\_URL**

HTTPS receiver endpoint ending /api/calendar/sync.

## **CALENDAR\_SYNC\_SECRET**

Shared random write credential on both server runtimes, never frontend.

## **NETLIFY\_AUTH\_TOKEN**

Optional legacy deployment credential.

## **NETLIFY\_SITE\_ID**

Optional legacy site identity.

## **VERCEL\_TOKEN**

Source deployment tooling credential, not data sync authentication.

## **VERCEL\_PROJECT\_ID**

Target project for direct deployment tooling.

## **VERCEL\_PROJECT\_NAME**

Deployment naming for the legacy/direct adapter.

## **VERCEL\_TEAM\_ID**

Scope deployment API calls to the owning team.

## **VERCEL\_ROOT\_DIRECTORY**

Optional direct-deploy root prefix; current standalone folder vercel-public.

## **BLOB\_STORE\_ID**

Vercel-connected OIDC store ID, public runtime server-side only.

## **BLOB\_READ\_WRITE\_TOKEN**

Alternative legacy Blob credential; do not expose to visitors.

## **NODE\_ENV**

Production/development runtime convention.

## **NODE\_OPTIONS**

Node flags such as heap limit; not a total-server RAM limit.

## Appendix D Command reference

```
npm ci
```

Repository root: install locked bot/admin dependencies.

```
npm start
```

Repository root: start HTTP, schedulers and optional bot.

```
npm run register
```

Repository root: publish global Discord commands using credentials.

```
npm run register:guild
```

Repository root: publish only configured test guild commands.

```
npm run assets:sync
```

Repository root: copy canonical shared assets to provider folders.

```
npm run format
```

Repository root: write consistent formatting, not runtime tests.

```
npm run format:check
```

Repository root: report formatting differences without writing.

```
npm run db:backup
```

Repository root: consistent new private database snapshot.

```
npm run db:backup -- ./backups/manual.sqlite
```

Explicit new backup destination; must not exist.

```
npm run calendar:export
```

Generate public snapshot only; no website source deployment.

```
npm run calendar:publish
```

Export then invoke configured provider, with real side effects.

```
npm ci --omit=dev
```

vercel-public folder or production root as appropriate.

```
git diff --cached --name-only
```

List staged filenames, not an upload.

```
systemctl --user status tutelbot.service
```

Homeserver: inspect active service; not repository template name.

```
journalctl --user -u tutelbot.service -n 100 --no-pager
```

Homeserver: recent logs; redact secrets before sharing.

```
systemctl --user restart tutelbot.service
```

Homeserver: restart actual production service; affects playback.

# Appendix E Database schema reference

## users

```
id INTEGER PRIMARY KEY,  
username TEXT NOT NULL UNIQUE,  
password_hash TEXT NOT NULL,  
role TEXT NOT NULL CHECK(role IN ('admin', 'viewer')),  
must_change INTEGER NOT NULL DEFAULT 0,  
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
```

## sessions

```
token_hash TEXT PRIMARY KEY,  
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
csrf TEXT NOT NULL,  
expires_at INTEGER NOT NULL
```

## events

```
id INTEGER PRIMARY KEY,  
guild_id TEXT,  
title TEXT NOT NULL,  
description TEXT NOT NULL DEFAULT "",  
starts_at TEXT NOT NULL,  
ends_at TEXT NOT NULL,  
all_day INTEGER NOT NULL DEFAULT 0,  
holiday INTEGER NOT NULL DEFAULT 0,  
recurrence TEXT NOT NULL DEFAULT 'none',  
reminders TEXT NOT NULL DEFAULT '[15]',  
created_by INTEGER REFERENCES users(id),  
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
```

## guild\_config

```
guild_id TEXT PRIMARY KEY,  
guild_name TEXT NOT NULL DEFAULT "",  
channel_id TEXT,  
default_reminder INTEGER NOT NULL DEFAULT 15,  
timezone TEXT NOT NULL DEFAULT 'Asia/Bangkok'
```

## reminder\_log

```
event_id INTEGER NOT NULL REFERENCES events(id) ON DELETE CASCADE,  
occurrence_at TEXT NOT NULL,  
offset_minutes INTEGER NOT NULL,  
sent_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
PRIMARY KEY(event_id, occurrence_at, offset_minutes)
```

## app\_settings

```
key TEXT PRIMARY KEY,  
value TEXT NOT NULL
```

## calendar\_deliveries

```
id TEXT PRIMARY KEY,  
guild_id TEXT NOT NULL,  
channel_id TEXT NOT NULL,  
event_key TEXT NOT NULL,  
occurrence_at TEXT NOT NULL,  
schedule_key TEXT NOT NULL,  
message_id TEXT,  
payload TEXT NOT NULL,  
active INTEGER NOT NULL DEFAULT 1,  
sent_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
UNIQUE(guild_id,channel_id,event_key,occurrence_at,schedule_key)
```

## calendar\_latest

```
channel_id TEXT PRIMARY KEY,  
message_id TEXT NOT NULL
```

events also gains reminder\_mode default offsets and color default #4285f4 through column checks before ALTER TABLE. Fresh tables pass these additions too. Editing DDL does not automatically migrate existing data.

## Appendix F Worked examples

### All-day / ทั้งวัน

Admin selects 13 October and submits Project meeting, allDay true and blue. POST checks session/admin/CSRF; saveEvent converts midnight to UTC and next-midnight exclusive end. Sync debounces, exports, hashes, uploads, validates and stores Blob. The browser sees it after cache/poll. Default standard schedules are noon on 12 October and 07:00 on 13 October if custom notifications are enabled.

### Weekly / รายสัปดาห์

A weekly lesson at 18:30-19:30 is one stored definition, not 52 inserted rows. Each query expands overlapping instances. ID+at distinguishes weeks and the link selects the chosen occurrence. Editing the base rule changes later query results.

### Concurrent work / งานพร้อมกัน

Radio streaming uses FFmpeg and a guild player while Express saves a calendar event. Async streams/network permit progress; synchronous DB/password work occupies Node briefly. Calendar edits do not replace radio. Bot toggle destroys audio players while web/calendar services remain.

### Failed upload / ส่งไม่สำเร็จ

The local event commit survives a failed POST. Public stays at last successful revision. Pending revision persists; retry reuses it for unchanged content. A later newer edit obtains a newer revision. Inspect last error rather than reverting the event just because the public page is old.

# Appendix G Glossary

## API

A contract of paths, methods, inputs and outputs between programs.

## HTTP method

GET reads; POST creates/acts; PUT replaces/updates; DELETE removes.

## JSON

Text representation of structured objects/arrays; not executable code.

## DTO / projection

Selected data shape crossing a boundary.

## Snapshot

A complete captured view at a point in time.

## Revision

Ordered version used to reject older writes.

## Content hash

Digest identifying meaningful data content.

## ETag

Representation identity used for HTTP validation/conditional writes.

## Idempotency

Retrying the same operation does not create an extra logical effect.

## Debounce

Wait until a burst of changes settles before doing work.

## Backoff

Increasing delay between failed retries.

## Eventual consistency

Replicas converge after propagation rather than immediately.

## Occurrence

One time instance of a potentially repeating event.

## Exclusive end

End boundary itself is outside the event interval.

## UTC / ISO

Standardized instant representation, separate from displayed timezone.

## Guild

A Discord server.

## Interaction

A command/button event sent by Discord to an application.

## Ephemeral

Discord response visible only to the invoking user.

## PCM

Uncompressed sample representation emitted by FFmpeg.

## Opus

Audio codec used in this Discord voice pipeline.

## Pipe

Stream connecting one process output to another input.

## Session

Temporary authenticated state, identified by a browser token.

## Cookie

Origin-bound browser value sent with matching requests.

## CSRF

Unwanted authenticated browser requests; mitigated by session-specific proof.

## Hash / salt

One-way digest; random salt distinguishes identical passwords.

## Foreign key

Database relationship enforced between records.

## WAL

Log containing committed writes before checkpoint into the main file.

## Blob

Stored object/file, not a relational row collection.

## OIDC

Identity mechanism allowing the deployed runtime to authenticate to storage.

## CDN

Edge response cache/distribution layer.

## PWA

Web app using manifest and browser capabilities for install/offline behavior.

## Service worker

Browser background script controlling selected network/cache behavior.

## Facade

Small stable interface exporting focused implementations.

## Composition root

Entry point assembling application services.

# Appendix H References and further reading

Implementation details derive from this repository. These primary references explain concepts and changing platform contracts. Diagrams are vector technical drawings of the system, not AI artwork or unlicensed external imagery.

## SQLite WAL

<https://www.sqlite.org/wal.html>

## SQLite VACUUM

[https://www.sqlite.org/lang\\_vacuum.html](https://www.sqlite.org/lang_vacuum.html)

## Node SQLite

<https://nodejs.org/api/sqlite.html>

## Discord interaction response contract

<https://github.com/discord/discord-api-docs/blob/main/developers/interactions/receiving-and-responding.mdx>

## discord.js API

<https://discord.js.org/docs/packages/discord.js/main>

## Discord voice package

<https://discord.js.org/docs/packages/voice/main>

## yt-dlp official project

<https://github.com/yt-dlp/yt-dlp>

## FFmpeg documentation

<https://ffmpeg.org/documentation.html>

## Express documentation

<https://expressjs.com/>

## Helmet project

<https://helmetjs.github.io/>

## Vercel private Blob

<https://vercel.com/docs/vercel-blob/private-storage>

## Vercel Blob OIDC

<https://vercel.com/changelog/vercel-blob-now-supports-oidc-authentication>

## MDN service workers

[https://developer.mozilla.org/en-US/docs/Web/API/Service\\_Worker\\_API/Using\\_Service\\_Workers](https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API/Using_Service_Workers)

## MDN PWA caching

[https://developer.mozilla.org/en-US/docs/Web/Progressive\\_web\\_apps/Guides/Caching](https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Caching)

## Tailscale Serve

<https://tailscale.com/kb/1242/tailscale-serve>

## Radio Browser API

<https://api.radio-browser.info/>

## date-holidays source

<https://github.com/commenthol/date-holidays>

## Prettier options

<https://prettier.io/docs/options>

Release note: this handbook documents this source edition, not acceptance of every audio source or completeness of every holiday. Consult the accompanying task summary for actual rollout status.

# Appendix I Discord command input contracts

/play query

Required title or URL; caller must be in voice. Metadata lookup then enqueue, not a guarantee of sound.

/queue

Shows current queued tracks; state is per guild in memory.

/skip

Skips current normal track; no current track means nothing to skip.

/stop

Clears queue/random/radio and stops sound; stays connected.

/pause /resume

Pause or resume existing player.

/nowplaying

Shows track/radio status implemented by handler.

/leave

Stops streams and destroys guild connection/state.

/randommusic

Caller voice required; continuous SoundCloud topic searches.

/radio list area

Optional area: bangkok/north/central/east/northeast/west/south/hatyai.

/radio play station area

Station required; frequency or name plus optional area. Ambiguous directory results request a precise name.

/calendar setup channel pin

GuildText channel and PIN required. Retains existing default offset.

/calendar add title starts

Both required; starts YYYY-MM-DD HH:mm Thai. Optional duration 1..10080 (default60), reminder0..10080 (guild default15), repeat and description<=500.

/calendar list

Upcoming thirty days, first ten expanded entries.

/calendar delete id

Positive integer ID required; deletion restricted to guild-owned custom event.

/calendar config reminder

Required 0..10080 default offset; requires channel setup first.

/calendar test

Sends a real simple readiness message to configured channel; differs from admin embed test.

Event colors and all-day creation are web form capabilities; current Discord add does not define color/all-day options even though storage supports them. Not every web capability has an equivalent slash option.

# Current operation • 4 October 2026

## Architecture, storage and publication

This supplement supersedes conflicting baseline behavior. Contents: architecture; music/radio/messages; calendar; control panel/operations.

The active cluster node owns Discord login, audio, calendar scheduling and outbound public-calendar publication. Homeserver is preferred in automatic mode; Oracle is the standby node. A renewable MongoDB lease and a local watchdog prevent an inactive node from continuing bot work.

MongoDB Atlas shares users, password hashes, sessions, events, settings and delivery records. SQLite remains supported for standalone installations. Audio streams, playback queues, radio-list sessions and private-reply references remain in process memory and are not resumed after restart or failover.

The private admin panel runs locally on each server. Authorized commands to the active node travel through encrypted MongoDB jobs, so an inbound public Oracle address is unnecessary. Admin sessions, role authorization and CSRF checks still apply.

Vercel public-calendar updates use the authenticated snapshot API and private Blob storage. The configured sync secret authenticates the sender. Public snapshots exclude authentication data. Adding an event sends data; it does not require rebuilding or deploying the public site. Homeserver IP changes do not affect outbound synchronization.

# Current operation • 4 October 2026

## Music, radio and Discord message lifecycle

/join joins voice and displays the existing shared player without starting audio. Five idle minutes with no queue, song, random playback or radio triggers leave. A new play request edits the same message even if requested from another text channel; leaving removes it. Restart does not restore audio or the in-memory queue.

The Random button always opens a private genre selector. Its 25 choices include mixed music and Japanese music. Selecting a genre while random playback is active keeps the current song playing, discards unsuitable preloaded random audio, and uses the new genre for subsequent random songs. User-queued songs still have priority. Disable random is inside this private menu.

The single Radio button opens a private station selector while in music mode. Select a station to validate it and prepare audio before replacing the music. The old queue and random preference are retained. Press the same Radio button while radio is active to return to music. An interrupted song restarts from its beginning; the queue is not a seekable recording. There is no separate Return to music button.

Radio list has ten station choices per page, region/type filters, search, page navigation, a refresh check and the last station. The player radio selector uses at most twenty choices per page. Only the opener can use the private selector, and playback requires the same voice channel as the bot. Station health is measured from decoded samples, not just HTTP status. A one-second silent sample may mean a quiet interval rather than an offline station.

Private interactive messages have a hard three-minute lifetime from creation; navigation does not extend it. Expired visible buttons are rejected and ask the user to reopen the menu. Text-only music/radio confirmations expire fifteen seconds after work finishes. The public music panel has no three-minute timer. Discord modals cannot be remotely closed by the bot. Graceful shutdown cleans private replies; a crash cannot guarantee timely message deletion.

Loop defaults to Queue and is stored per guild in `music_loop:<guildId>`. Settings offers Off, Song and Queue; the main Loop button also cycles them. Stop clears playback and random mode but retains the Loop preference. Queue-loop Skip moves the skipped song to the tail; song-loop Skip bypasses the repeat. Random mode does not recycle the queue indefinitely. Live radio has no Loop.

Unknown YouTube duration originated in the fast direct-link oEmbed path, which supplied title and image but left duration at zero. Missing durations are now hydrated for the selected track with yt-dlp full metadata. Known durations and live streams avoid this lookup. Playlist entries with unknown duration are hydrated only when selected for playback or preparation, never all at once. Lookup has a six-second deadline, a five-minute positive cache and a 128-entry bound. Failure retains playback with an honest unknown duration; cancellation still prevents playback.

The reproduced Jeff Satur Ghost link `qguo-j5PxBE` returned zero from the old direct path, 250 seconds from full metadata, and approximately 251 seconds from flat search. The current full metadata result is 4:10. Provider search and full extraction may differ slightly; the UI must not guess duration from downloaded byte size or

elapsed playback.

homeserver and Oracle use Smooth transition by default when the guild has no saved choice; saved 0 remains disabled. The repository fallback remains off for new standalone installations. Smooth/full compressed preloading is eligible only for tracks with a known finite duration above zero and at most 600 seconds. Longer, live and still-unknown tracks use ordinary streaming. Only one successor is prepared, with a 32 MiB compressed cache cap and 350 ms crossfade. Resource preparation and network problems can still cause waiting.

The bot-profile Commands showcase is controlled by Discord. /join is a normal global slash command; the supported application-command API does not expose a setting that pins only /join to this showcase. No other working commands are deleted to influence the profile. This remains a platform limitation, not a completed profile customization.

# Current operation • 4 October 2026

## Calendar notifications and command visibility

One automatic agenda is delivered per configured guild/channel and Thai date at dayTime, default 07:00 Asia/Bangkok. Category and nonholiday filters apply. Empty days send nothing. Event edits update the recorded daily message. Long agendas include a complete text attachment.

This daily summary replaces the earlier automatic per-event and previous-day reminder schedules. Legacy offsets remain metadata and do not generate additional automatic messages. Restart catch-up is for today only; database/Discord deduplication is best effort because no cross-system transaction exists.

Calendar buttons show a private link to the correct date/event for thirty seconds. Only the latest notification keeps its calendar button. Calendar slash commands are registered per guild only when its notification channel is configured and notifications are enabled; configuration is checked every thirty seconds.

Admin event editing and the public read-only calendar use the same published event data. The public frontend is separate from the private admin panel. Event category, color, all-day/timed boundaries and recurrence are retained in the calendar model.

# Current operation • 4 October 2026

## Control panel, logs, operation and verification

Bot controls show every connected guild, its current track or radio station, voice channel, queue length and random mode. Playback states distinguish disconnected, standby, loading, music, radio and paused. The visible admin page refreshes every five seconds; background tabs and focused music forms pause this refresh to avoid unnecessary work and losing edits.

Host cards distinguish the selected mode from the currently active lease owner. The active badge remains visible on mobile. Selecting the already-selected card sends no request. Pending switches disable the other choices. Changing the host while the bot is disabled preserves its disabled state. Switching hosts loses the in-memory queue and playback.

Logout is at the end of sidebar navigation, with a compact labeled icon on mobile. Control/settings/log pages use a bounded reading width, spaced cards and theme-aware controls. Calendar layout is preserved.

Log access requires an admin session. The browser loads `/bot-logs.js`, served explicitly by `assets.js`, then requests `/api/bot/logs`. The missing asset route caused the former empty screen; it now returns JavaScript successfully. The visible log page polls every three seconds, supports pause/refresh/clear and resets on instance changes. Clear affects the display, not server history.

Log history is bounded to 500 lines per process, 2,000 characters per line and 200 entries per response. Known secrets are redacted. A restart resets this in-memory history; use `tutel log` for journald history. Forwarding follows the active cluster node.

Both servers use the user service `tutelbot.service`. The `tutel` shortcuts offer `start`, `stop`, `restart`, `status` and `log`. Deploy only the files required for an approved task, keep backups and preserve real environment files. GitHub pushes are separate from server deployment and require the user instruction. This update passed 45 automated tests and verified live control status and nonempty logs in the homeserver browser.

# Current operation • 4 October 2026

## Smooth transitions: continuous audio and RAM preparation

The private music-panel settings have separate Enable and Disable buttons. Manage Server and voice checks apply. Repository fallback off; live homeserver/Oracle fallback on; stored per guild as music\_smooth:<guildId>. The preference persists, but queues and audio do not survive restart or failover. Live radio is unchanged.

As a song starts, player.js chooses one successor from queue, loop or random discovery. stream.js downloads complete compressed audio into RAM, then FFmpeg prepares bounded 48 kHz stereo PCM. No media file is saved; network bandwidth is still consumed.

continuous-pcm.js keeps one PCM stream, one Opus encoder and one Discord AudioResource. It mixes old and new audio for 350 ms with opposing linear gains at natural endings and Skip. After handoff, preparation of the next successor starts. The overlap shortens total duration slightly.

The RAM cache is capped at 32 MiB per song; concatenation can briefly double its memory cost. Over-limit songs fall back to bounded streaming. Preparation timeout is 90 seconds and decoder readiness timeout is 25 seconds. One successor per guild means simultaneous guilds multiply memory and process costs.

If Skip occurs before preparation is ready, the old song keeps playing. A natural ending may still wait for a slow source. Enabling during playback migrates the decoder once and may briefly interrupt it. Leading quiet audio and retained digital tail silence are trimmed, not intentional pauses throughout a recording.

Stop, Leave, radio and stale preparation cancel work. Inspect fully cached, bounded stream and continuous transition logs. Synthetic natural-end and Skip checks found no silent PCM/Opus test frames and no player resource replacement between songs. A real YouTube item decoded nonzero samples on homeserver; this is not proof of listener delivery under every network condition.

Text-only music/radio acknowledgments and errors auto-dismiss 15 seconds after handler completion. Deferred progress remains during work and interactive menus keep their normal lifetime. The public player is unaffected. The promotional site now presents music, radio and smooth transitions; the personal calendar remains a separate application concern sharing this runtime.

Only known, finite tracks up to ten minutes use full successor preparation. A saved guild disable choice is respected on the two live hosts.